GREEN ALIBI — Development Log

This is my working log for GREEN ALIBI. I am testing whether Solar-Induced Fluorescence (SIF), which is tied to photosynthesis, catches crop drought stress earlier than NDVI, the greenness index that India's drought monitoring relies on. The entries are in the order I wrote them, one per phase of the work: framing the research question, building the SIF and NDVI pipelines, fixing the study boundary, bringing in rainfall, adding a second lag method, reviewing my own code, extending the study from 3 years to 9, and finally to an out-of-sample test on 2001–2014. Each entry records what I found and decided at the time, including results that did not support my hypotheses and errors I later corrected.

> **Note on numbers.** Entries are kept as written at the time, so earlier entries contain numbers that were later corrected or reinterpreted. Entries 24–27 describe the current state: the seasonal lead depends on how each curve is normalised and, for GOSIF, is shared with NIRv; the methods were tested unchanged on 2001–2014; district drought is classified against each district's own rainfall baseline (15 drought district-years, not 17); Trigger-1 dry spells are counted only within the monsoon; and the district alarm comparison was corrected in Entry 27, so the district alarm counts in Entries 24–26 are superseded. Where an earlier entry disagrees with Entries 24–27 or `GA_Research_Paper.md`, the later one holds.

Index

1. Entry 1
2. Entry 2
3. Entry 3
4. Entry 4
5. Entry 5
6. Entry 6
7. Entry 7
8. Entry 8
9. Entry 9
10. Entry 10
11. Entry 11
12. Entry 12
13. Entry 13
14. Entry 14
15. Entry 15
16. Entry 16
17. Entry 17
18. Entry 18
19. Entry 19
20. Entry 20
21. Entry 21
22. Entry 22
23. Entry 23
24. Entry 24
25. Entry 25

26. Entry 26
27. Entry 27

Entry 1

I am running the GREEN ALIBI project as an independent satellite study. My main goal here is to test if Solar Induced Fluorescence—which works as a direct physical proxy for actual photosynthetic activity—can catch crop and vegetation drought stress much earlier than the standard vegetation greenness index called NDVI that India's official drought monitoring setup currently uses. I want to see if the government is missing early warnings. See, NDVI is just a basic reflectance based index. It measures how much visible and near infrared light a leaf's skin reflects back into space, meaning that reflection only changes once a plant's canopy has already started thinning out visibly. By then, much of the damage is done. I chose to use SIF because it tracks a physically earlier, more direct signal inside the leaf. When a plant absorbs sunlight for photosynthesis, a small piece of that energy is not changed into food but is instead shot back out as fluorescence, a process decided by the quantum yield of photosynthesis. When a plant gets hit by heavy physiological stress, its internal efficiency drops way before its outside greenness changes, and this drop shows up on my satellite data as a quick change in the fluorescence emission well before NDVI shows even a small dip. That was the idea I set out to test: NDVI can give a stressed crop a green alibi, while SIF should show what is happening inside the leaf.

I am looking closely at India's drought declaration rules. The big issue is that government crop insurance cash payouts under the Pradhan Mantri Fasal Bima Yojana scheme depend fully on rainfall deficits and old NDVI based satellite checks. This setup creates a bottleneck. Both tools are just reflectance or rainfall indicators that only flash red after the field crops are already dying visibly. Delayed recognition means delayed insurance money, and farming families suffer heavily because they need this cash fastest right when the heat hits. I am treating this delay as a real, empirical testable question rather than just assuming it blindly. My main data goal is to check if SIF really catches physiological plant stress way before NDVI, and if that time lag is big and consistent enough to matter in real life. If the data showed this, it would be a physics based case for updating the warning tools used in national policy. If they did not, that would be worth knowing too.

So I set a clear goal for my study. I am using satellite derived SIF and NDVI datasets over drought affected farming districts in India to independently test if SIF shows a measurably earlier drop than NDVI during real drought episodes. I want to calculate the exact timing difference and check if the resulting time lag is actually large enough and consistent enough to help people with early warning value for drought declaration and crop insurance timelines. This broke down into 4 clear questions.

1st, I want to find out if SIF drops measurably earlier than NDVI during documented drought onset periods in my study region (RQ1). Then, I look at the next part. So I want to see if this SIF–NDVI lag is identical across different districts and crop types, or if it varies heavily based on geography and crop physiology (RQ2). It could be random. After that, I run a government comparison. I am going to check how the timing of SIF based plant stress onset compares against official state drought declaration dates for the same districts and years (RQ3). Finally, I look at the insurance angle, and I will calculate if the observed SIF–NDVI time lag gives a practically meaningful early warning window—measured in raw weeks—that is relevant to real crop insurance and drought relief timelines (RQ4). This will complete my tracking loop.

Had 3 main guesses before starting. 1st, I expected that SIF would show a statistically significant drop well before NDVI during the same dry spell, matching my theory that fluorescence catches a drop in internal photosynthetic efficiency long before you see any outward, reflectance based leaf color loss (H1). Need to test

this pattern across the board. 2nd, I look at crop differences. I guessed that the total size and exact timing of this SIF–NDVI divergence would not look uniform across crop types because fluorescence behavior depends heavily on unique canopy shapes and species specific photosynthetic pathways (H2). It could vary a lot. Finally, I have a government timeline guess. I expect that official state drought declarations will lag far behind my satellite SIF stress signal because the official bureau framework relies blindly on slower responding rainfall gauges and greenness indexes instead of checking direct biological plant health metrics (H3). Will check all 3 assumptions against my script outputs when the numbers come in.

So I am starting my earliest planning stage now. My 1st big job is to choose a proper study area with a very clear, well documented history of recent dry spells and reliable government files, so I am looking at candidate zones like the Marathwada and Vidarbha regions in Maharashtra or the Bundelkhand area that spreads across Uttar Pradesh and Madhya Pradesh because they all suffer from regular, heavily reported droughts. This will give me a good baseline. Next, I need to hunt for a usable SIF dataset... I plan to source either the gridded GOSIF product or a proxy layer derived from TROPOMI because the native fine resolution SIF space instruments are simply not built for tracking tiny, individual agricultural fields anyway. This is a tricky data task.

Then I have to solve the greenness data track. Must find matching NDVI data from MODIS or Sentinel-2 satellites for identical district boxes and time windows. Finally, I need to track down accessible official drought declaration paperwork and any PMFBY insurance payout timing records from local bureaus to use as my real world policy benchmark for my final code comparison. Have zero files in my project folder right now, and this log entry simply marks the absolute beginning of my data collection process and I have not downloaded or achieved any final outcomes yet. I need to start writing my download scripts tomorrow.

Entry 2

I am moving to the next task now. Following my project framing back in Entry 1, my next job was to build the actual working satellite pipeline by finding SIF data, sourcing a matching NDVI dataset, and finally producing my 1st real data comparison for Marathwada. This was a heavy coding phase. I finalized my study area and years. So I chose Marathwada as my primary study region — this was a deep personal choice as much as a methodological one because it is my own home region and everyone knows its well documented recent drought history... I feel connected to this data.

Selected 3 distinct years for my timeline. I chose the season from June 1 to November 1, which means day of year 153 to 305 covering the main Kharif growing season. I picked 2015 and 2018 because both are heavily documented Marathwada drought years, with 2015 in particular being the bad year of the severe Latur water crisis when trains had to carry water. It was a terrible time. Then I chose 2020 as my last tracking year. This was a comparatively normal monsoon year that I will use as a clean baseline for my final script comparisons. Timeline's locked in, moving on.

I found my SIF data source. Picked out the GOSIF v2 dataset, which is a global 0.05-degree 8-day solar induced fluorescence product made from OCO-2, MODIS, and weather reanalysis models by Li and Xiao at the University of New Hampshire. It is the most practical choice. Also I chose this because native space instruments are not built for tiny farm monitoring anyway. Downloaded 60 8-day GeoTIFF files in total — grabbed exactly 20 files each for my 3 target years 2015, 2018, and 2020 to cover my main growing window timeline. The download took some time.

Clipping and scaling.

So I wrote a python script to crop each global file. I set the script to cut the maps exactly to a Marathwada bounding box between 17.5°N to 20.5°N and 75.0°E to 78.5°E so it covers all 8 of my home districts perfectly. This step was critical. I had to multiply the raw digital values by a scale factor of 0.0001 to convert them into real physical SIF units. Then I had to explicitly mask out 2 specific fill value codes where 32766 means water bodies and 32767 means non vegetated or missing data blocks before running any calculations. Skipping this step would have ruined everything. These dummy background numbers are numerically huge and sitting far outside the real SIF range, so a naive average would get really corrupted by them. Coded a 2nd script after that. Forced it to aggregate each clipped image into a single district wide mean SIF number per date, and I also recorded the fraction of valid non masked pixels for every single time step as a data quality sanity check. Data looks clean on my screen now.

Hit a serious cloud masking bug in my NDVI code. I sourced my NDVI data separately using Google Earth Engine from the MOD09Q1 MODIS 8-day surface reflectance product because I wanted its temporal resolution to match my GOSIF 8-day timeline cadence exactly. My 1st script version was wrong. I calculated NDVI directly from raw reflectance bands without applying any cloud quality bitmask filtering lines, and the resulting NDVI time series came out implausibly noisy on my screen. During the 2020 season, my numbers swung wildly from roughly 0.55 down to 0.14 and back up to 0.60 within consecutive 8-day windows. This rate of change is physically impossible for agricultural cropland crops. I compared this jump against my SIF data series. My SIF line was smooth all the way through and made complete phenological sense, which proved to me that my NDVI script was the real problem, not the actual plant health signal. So I found the exact root cause quickly. The error came from heavy cloud and cloud shadow contamination because Marathwada's growing season hits right during the monsoon, and the raw MOD09Q1 product does not reliably exclude bad cloud pixels from its best observation window without explicit quality band filtering code. This messed up my baseline.

So I fixed my code. I rebuilt my entire NDVI extraction script around the MOD13Q1 dataset, which is NASA's own 16-day Vegetation Index product that runs a Maximum Value Composite algorithm specifically to kill cloud noise. It runs much cleaner now. This file comes with a SummaryQA quality band where 0 means good data, 1 is marginal, 2 means snow or ice, and 3 is cloudy, so I used this band to explicitly mask out everything except the good to marginal pixels. Then I added 1 more block. I chose to use a cropland mask using the MCD12Q1 land cover product with IGBP classes 12 and 14. I did this step so that random forest, urban buildings, and barren rock pixels inside my district boxes would not dilute my crop stress data signal. This step cleaned up my averages nicely. And I chose to trade my old 8-day time resolution for a slower 16-day one — this trade produced a visibly smooth, physically sensible NDVI curve across the chart, which was worth it because a clean 16-day trend line is far more trustworthy than a noisy 8-day one. Pipeline's running smooth.

Hit a 2nd, smaller bug. My updated script suddenly crashed inside Google Earth Engine with an Image.clip: Can't transform (0.0,0.0) error string because I was trying to run a .clip() command on a combined image made of 2 really different native resolution layers. One was 250m NDVI and the other was a 500m derived cropland mask grid. It was an annoying error. So I fixed it easily by removing that redundant .clip() line from my code entirely, since the reduceRegion() function already restricts its calculation to the supplied geometry boundaries anyway. The clip step was useless and was the exact thing throwing the error. This deleted the bug.

Merging 2 time series at different temporal resolutions.

Then I had to merge my timelines. Because SIF has an 8-day rhythm and my new cleaned NDVI uses a 16-day cadence, the 2 tracks did not share an identical date grid on my spreadsheet. Had to change my join logic.

Chose to link the 2 series using a nearest date matching loop per year instead of forcing an exact date join, so each 8-day SIF value gets paired up nicely with its closest available 16-day NDVI number on the timeline. The combined data frame finally looks right.

I caught a bad labeling bug. An early version of my comparison plot accidentally labeled 2018 as a normal monsoon year on the chart, when it is actually the 2nd of the 2 documented drought years in my project folder. It was an embarrassing mistake. This was just a simple conditional logic oversight in my code because only 2015 had been explicitly flagged as a drought year inside the plotting code block. Next I noticed the text error and corrected it before I drew any interpretations or conclusions from the mislabeled chart. This kept my figures accurate.

Ran my 1st seasonal comparison. With my corrected coding pipeline ready, I plotted the SIF and NDVI trajectories across all 3 target years to check the trends. A visual pattern holds everywhere. SIF always begins dropping from its maximum seasonal peak well before NDVI does anything. My data shows that NDVI holds near its top peak values for a noticeably longer window of time after my SIF line has already started crashing down toward zero. This is a real, replicated pattern, and it directly supports my core premise that SIF responds faster to post peak biological changes than old NDVI does. But I must fix my own overclaim here.

Initially made a lazy mistake. When I just eyeballed the 3 charts carelessly at 1st, I claimed that this time lag was distinctly larger during the 2 bad drought years 2015 and 2018 than in the normal 2020 monsoon year. So I chose to check this claim properly. I calculated each year's drop as a flat percentage of its own peak value at every single subsequent date on the calendar. My strict math did not support my initial guess. The real time lag stayed at a broadly similar size of roughly 15 to 25 days across all 3 years alike. Drought did not amplify the lag gap. I am retracting that bad overclaim right here in my journal rather than leaving a fake story standing in my log. My correct, defensible finding is that SIF leads NDVI's decline in general. If I want to prove whether drought specifically changes this time gap, I will need to calculate a proper quantitative lag metric like a day of year threshold math test or a formal cross correlation script instead of just looking at curves. That is my next coding task.

Status after this entry.

So I finished building both the SIF and NDVI extraction pipelines. Successfully cloud/quality screened and cropland masked all my raster grids, covering 3 full years of data from 2015, 2018, and 2020 over the Marathwada region. The files are fully clean now. I observed a real, replicated SIF leads NDVI pattern across my timeline charts. But I cannot report this as a final finding yet. I absolutely need a rigorous, numeric lag calculation script to take the place of the lazy visual comparison I used so far — that is my main priority. I also ideally want to download at least 1 additional comparison year to strengthen the small sample size my analysis currently rests on.

Entry 3

My 2nd log entry ended with an unresolved problem. Ran my very 1st quantitative lag calculation script—comparing how many calendar days after its own maximum peak SIF and NDVI each crossed a series of decline thresholds—but my final results were entirely untrustworthy. The core issue was my tracking window. My chosen observation window from June 1 to November 1 ended way too early before NDVI had even finished declining in 2 of my 3 working dataset years. It was a bad code boundary mistake. Several threshold markers simply never resolved at all in the console because they returned zero crossing dates. The few thresholds that did resolve properly were being averaged very unevenly across different timeline years, which

made any comparison between my drought and normal years unreliable rather than just being inconclusive. So I had to scrap that 1st math check entirely.

Chose to widen my timeline window. My fix was to extend the data collection much further into the year by adding 7 extra 8-day GOSIF periods covering day of year 313, 321, 329, 337, 345, 353, and 361 for all 3 years. This pushed my observation cutoff from November 1 out to late December. My old scripts handled this fine. The same clipping and aggregation scripts I built back in Entry 2 picked up these additional 21 files without requiring a single line of code change. Also I got lucky here. My logic was written to scan for whatever files exist inside the raw data folder rather than assuming a fixed date limit. Then I also extended my NDVI extraction script's end date from November 1 straight out to December 31. I re ran the same MOD13Q1 cloud screened, cropland masked pipeline for this longer window. Timeline problem's sorted.

My timeline extension worked perfectly. With this longer window, both my SIF and NDVI seasonal curves now taper down smoothly through day 350 to 360 in all 3 years on the chart. The ugly noise and sharp data truncation are gone, and more importantly, every single threshold from 90%, 80%, 70%, 60%, down to 50% of the seasonal maximum peak now resolves to an actual crossing date for all 3 years. This is a big win. My earlier problem of unresolved, missing thresholds is fixed, so I can now run year to year comparisons on a fair, equal footing. My data matrices match up beautifully now.

My final results shocked me. Once I made the timeline comparison fair across the board, my final data did not support my original guess or hypothesis as I had framed it earlier. My 1st hypothesis failed. I found that the mean lag across all 5 thresholds came out to 21.6 days for 2015 which is a drought year, 8.4 days for 2018 which is also a drought year, and a big 28.5 days for 2020 which is my normal monsoon comparison year. When I averaged them by group, the 2 bad drought years showed a much smaller mean lag of 15.0 days than my single normal year which sat at 28.5 days. This is the exact opposite of what my H1 theory predicted. This surprise is not an artifact of my older code truncation problem. With my newly extended data window, 2020's time lag at the 70%, 60%, and 50% thresholds forms a smooth, consistently large trend curve at 36.9, 38.9, and 39.5 days respectively on the chart. It was not an anomalous spike. A single random spike is exactly what you would see if this were still a bad data window error bug. The result looks real, not a fluke.

I hit an additional complication. My calculations showed that the 2 bad drought years do not behave alike at all because 2015's mean lag sitting at 21.6 days is roughly 2.5 times bigger than 2018's small 8.4 days. This happens despite both being fully documented Marathwada drought years. This wide gap tells me something clear. Treating "drought year" as a simple binary yes or no label is way too coarse a category for this deep analysis, and it hides the real physical nuances. The 2 droughts definitely differed in their starting onset timing, total severity, or overall duration in ways that matter deeply for this specific comparison. My current dataset is just too small. A tiny sample of only 2 drought years is nowhere near enough to characterize that variation, let alone average over it in a meaningful way. So I need a lot more timeline entries to fix this gap.

My core finding still holds up. Across all 3 years—drought and normal alike—I found that SIF consistently begins its post peak decline before NDVI does anything, with NDVI holding near its peak value for a longer stretch after my SIF line has already started dropping. This specific part of the signal is very robust. Replicated it 3 times over in my scripts. This pattern directly supports my project's main premise because it proves SIF is a more temporally responsive proxy for the starting onset of physiological decline than NDVI is. But my data does not support everything. At least not with this current sample, it fails to support the more specific claim that drought stress heavily amplifies this time lag relative to a normal crop season. I have to accept this fact. If anything, the limited evidence available in my folder points the other way, though with only 3 years consisting of 2 droughts and 1 normal season, this is far too small a sample to treat as a confident finding in either

direction. And I am just calling it a non result on the drought amplification question, not a confirmed contrary finding.

Status after this entry.

The SIF leads NDVI pattern holds up well at this point. I calculated this across 3 separate replicated observations for Marathwada, stretching right across the full growing season all the way through late December. Tracking sheet's in good shape now. The specific guess that drought years always show a much larger SIF–NDVI time lag than normal years is unsupported by my data sheets. So I found that the 2 dry years differ from each other by a huge margin... In fact, they differ by more than either differs from my single normal monsoon year, which really complicates any lazy binary drought or non drought framing for this comparison. The result here is a clear replicated SIF leads NDVI pattern with an inconclusive relationship to actual drought severity at this tiny sample size.

Entry 4

I finished my main timeline calculations. After I had a complete, extended window lag analysis safely behind me running from June to December across 2015, 2018, and 2020 with all 15 threshold year combinations fully resolved, I moved straight onto the big coding task I had been putting off for days. So I had to look at maps. I chose to actually examine my SIF data spatially instead of only looking at it as a single whole region average number on my spreadsheet. I built my very 1st mapping script called map_sif_spatial.py to plot the raw clipped SIF rasters for a single day of the year at day of year 273, which is right around the exact point where the seasonal decline is well underway in the fields. I forced the program to display the maps side by side for all 3 tracking years using a strictly shared color scale so my comparison stays fair.

My 1st map really looked good. I saw a visibly concentrated low SIF patch marked in dark red sitting right in the west southwest corner of my study area during the 2015 and 2018 grids, while the 2020 panel looked way greener. This is roughly where I would expect drought stress to pop up if my core theory carries any spatial coherence and not just a temporal one. I am testing whether SIF flags drought earlier and more precisely than old NDVI, so seeing this clear shape gave me high hopes. But then I stopped dead.

Looking closely at that map made me ask a basic question. Realized I should have asked this much earlier in my logs: what exactly are my study area boundaries? Had skipped this check. Up to this point, every single script in my project directory—my GOSIF clipping code, my NDVI extraction functions inside Google Earth Engine, all of it—had been built around 1 basic rectangle. I set the geographic limits to 75.0–78.5°E and 17.5–20.5°N right at the start because it seemed roughly like Marathwada's location on a map. I nvr went back to double check those numbers against any precise official shapes, just eyeballed a general map early on and blindly trusted it for months. Now I had to fix this loose end.

So I opened the Earth Engine Code Editor. I finally typed in the 2 lines that I had nvr bothered to add before—Map.centerObject() and Map.addLayer()—just so I could draw that rectangle on my screen and check it properly. So I zoomed right in on the map. The mistake was obvious. While my box covered all 8 real Marathwada districts, it also swallowed a massive chunk of Telangana including parts of Nizamabad, Bidar, and Adilabad, touched Solapur, and clipped a corner of Yavatmal. None of these places belong to Marathwada, and they also do not share my region's drought history because Solapur and the Telangana districts sit in entirely different rainfall regimes. This was a bad data leak. Every mean SIF and mean NDVI number I calculated up to this point had quietly included a huge slice of somewhere else.

So I chose to fix it immediately. I could not just let this go because the claim that this is a Marathwada specific finding is central to my entire project logic. A rectangle is a bad idea anyway, since Marathwada has a very irregular shape on the ground with Nanded pushing out far east and Osmanabad pushing deep south, and messing with coordinates would not help. Next I decided the correct fix was to drop using a box and switch to actual official district lines instead. This was the only right way forward.

So I switched to Earth Engine's FAO GAUL 2015 level-2 administrative boundaries dataset. I filtered it down strictly to Maharashtra state and picked the 8 district names that make up Marathwada, then merged them into a single precise polygon to replace my old rectangle everywhere in my NDVI script. It was a necessary cleanup pass.

But I immediately found a bug in my own fix. The very 1st time I ran my updated code, I noticed a huge visible gap right where Beed district should have been on my screen. The rest of the map filled in fine. So I had filtered using the modern district name 'Beed' in my text string, but the old FAO GAUL 2015 dataset stores this area under its older spelling 'Bid', so my code found zero matches. It was an annoying silent failure. The script ran with zero errors but quietly dropped the district, giving me 7 shapes instead of 8. I only caught it because I looked at the map with my own eyes rather than trusting the terminal's silence. So I fixed the spelling string immediately, and also added an explicit count check using the .size() function right after my filter loop to print out the number of matched districts. Now, if any dataset naming quirk acts up again, my script will flash a wrong number right on my screen instead of hiding a bad map shape. This makes my check safe.

Fixed my NDVI boundaries 1st. I matched everything perfectly against an academic reference map of Marathwada's district shapes that I discovered online, but I still had the heavy SIF side left to repair... My code had a huge systematic design gap. My old GOSIF clipping script was running rasterio's window based cropping layout, which is inherently rectangular. A basic rectangle window has absolutely no way to represent or trace an irregular shape polygon boundary like real state borders. It was a clumsy design setup. Exported the precise Marathwada boundary shape right out of Google Earth Engine as a small GeoJSON file to fix this mess. I brought it straight into my local project directory and fully rewrote clip_gosif.py to use rasterio.mask.mask() against that polygon boundary and not using a loose box window. This step fixed my averages. Every pixel that sits outside the real Marathwada district outlines is now set to a blank NaN value, which removes all the unwanted data that used to leak into my averages from Telangana, Solapur, and Yavatmal. Glad my regional averages aren't contaminated anymore.

So I re ran my full pipeline chain, triggering clip_gosif.py, then aggregate_sif.py, then compare_sif_ndvi.py, and finally lag_analysis.py sequentially on my local terminal machine. I processed all 4 scripts in order using my newly corrected shape files, running this heavy pass for both datasets across my entire folder timeline to wipe out the old coordinate calculations. My outputs match up nicely now.

Was bracing for bad changes, since the new numbers could have flipped my conclusion. But that is not what happened at all. My quick sanity check SIF means by year came out to 0.137 for 2015, 0.151 for 2018, and 0.219 for 2020, and these fresh values preserved the identical drought versus normal ordering pattern that I found before cleaning the folder. Trends check out fine. The seasonal SIF vs NDVI curves still showed SIF's decline consistently preceding NDVI's drop across all 3 target years. Also, my time lag numbers told the same story even though they were not identical to my older contaminated box run. Checked the calculations carefully on my screen. Found the mean lag across thresholds was 24.2 days for 2015, a small 5.1 days for 2018, and 25.5 days for 2020. This means my drought year average of 14.6 days stays much smaller than my single normal year's 25.5 days gap, so my H1 hypothesis is simply not supported again. If anything, the gap

between my 2 separate drought years 2015 and 2018 came out even more pronounced with this cleaner geometry than it had with the messy, leaked rectangle. The cleaned up numbers back this up too.

So I am treating this as a truly useful result. Getting the identical qualitative answer from 2 really different, independently built definitions of my study area—a rough rectangle box and a precise administrative shape boundary—is a solid check on whether my earlier trend finding was a real signal or just an artifact of sloppy data clipping. It held up. This does not make my drought amplification hypothesis correct at all. But it does mean I can now say with much higher confidence that rejecting my H1 guess is not a coding mistake caused by including the wrong pixel rows in my means. The math behind this checks out.

So I still have some big gaps. I do not have any ground truth confirmation that the specific low SIF red patch on my spatial map actually lines up with farm croplands rather than some other random land cover class sitting inside those same mixed pixels. Also, my sample size stays tiny — the timeline is still stuck at only 3 years of data, which severely limits how far I can generalize any of these findings to other decades. Both of these gaps stay on the record.

Set my immediate next step. I need to regenerate my spatial SIF map layout using my newly corrected clipped rasters because my boundary fix scripts already overwritten the underlying files anyway. This should be a quick re run. Just need to trigger my old script execution line again instead of rewriting any new python code blocks, and then I will move straight onto building the interactive Folium version for the dashboard. Ready to keep going from here.

Entry 5

I finished my boundary correction step. And I re clipped my GOSIF layers with the real official 8 district polygon shape rather than using that rectangle and re extracted my NDVI data the same way through Earth Engine. Next, I wanted to see the new maps. I chose to regenerate my seasonal spatial comparison map for day of year 273 across 2015, 2018, and 2020 because I fully expected it to show Marathwada's actual irregular outline on my screen instead of a plain boring box. But my code failed.

The new map did not change. When I reran my map_sif_spatial.py script on my terminal, the final output image still looked like a smooth, fully colored giant rectangle with zero differences in shape compared to my older bad run. This outcome was highly worrying. So I realized I was facing 2 very different problems — either my district boundary correction step hadn't actually changed anything inside the raw raster data grids, or the clipping worked perfectly but my plotting script was failing to show the transparent mask edges. So I did not want to guess blindly. So I had to look into the raw arrays next.

Then I stopped trusting my pictures blindly. Rather than guessing whether a good looking or bad looking image map on my screen was right, I chose to build a small, separate python diagnostic script to answer my questions directly. Forced the script to open a single clipped SIF data layer, plot it cleanly on screen, and draw my actual official Marathwada boundary polygon shape as a bright blue outline right on top of the matching coordinate workspace axes. It gave me an easy visual test. If the colored raster cells spilled out past that bright blue border line anywhere on the grid, my clipping code had obviously failed. But if the pixels cut off exactly at the blue line edge, it proved my masking logic was 100% correct and my old rectangle layout impression was coming from a different bug. I triggered the script immediately.

The overlay map came back perfectly clean. My SIF data pixels cut off exactly at the boundary line everywhere on the grid, matching all the tight smaller notches and even the complex eastward hook shape sitting right near Nanded. This was a relief. It proved that my heavy shapefile clipping fix from my last journal entry was

really working on the raw grid arrays. The underlying data files inside my folder were correct, and my actual pipeline bug was hiding inside the map plotting script itself instead of the raster layers. Knew where to hunt next.

I looked at my script files again. I opened map_sif_spatial.py and found that while I had fixed clip_gosif.py to mask my raster data nicely against the real district shape file, I forgot to go back and update my main multi panel plotting script to match. This oversight made the display misleading. The drawing script was still blindly loading each raster grid using a static, fixed rectangular scale limit left over from my older box based days, and it never actually plotted the district boundary outline lines anywhere on the panels. It was a silly logic lapse. Even though my underlying data array held the correct NaN pixel mask patterns outside the real district shapes, nothing in my plotting code was actually telling the system to make those transparent areas visible to my eyes across 3 side by side charts. It just filled up the blanks.

I chose to fix the code layout. I fully rewrote the python script to force the tool to pull each raster image's true geographic boundaries directly from its metadata dictionary rather than using a hardcoded rectangle. This change worked immediately. I also added lines to draw the official boundary polygon outline on every single chart panel right on top of my shared color scale and the panel spacing formatting patch from earlier. This gave me an excellent clean view. Also chose to rename my output plot file with a fresh version suffix code string. I did this and not silently overwriting my old image file in the directory, matching the same tracking habit I keep using for all my Google Drive exports. This saves me from massive confusion later about which picture is my corrected one.

My new regenerated figure finally shows the correct map layout. All 3 chart panels now accurately display Marathwada's real, irregular border outlines with the raster pixel data confined exactly within the district edges. The core biological pattern itself held up perfectly under this more careful map rendering. Found that 2015 and 2018 both show a very distinct, spatially coherent low SIF drought zone heavily concentrated along the western edge of Aurangabad district and running down straight through the Latur–Osmanabad farming belt. This makes total sense. On the other hand, my 2020 map panel comes out overwhelmingly high SIF green across almost the entire region, though there is only a tiny, minor patch of lower stress values sitting way down in the south of my plot. My comparative results look solid.

Learned a process lesson from this. A chart figure looking visually plausible or nice on a screen is absolutely not the same thing as it being fully verified. Have to change my workflow mentality. I easily could have looked at that 1st rectangular version, lazily decided it was probably fine since the underlying processing pipeline was fixed, and just moved onto the next step. The old picture wasn't obviously broken or flashing red errors, it just simply wasn't showing what it should have been showing. It was a hidden trap. Building that standalone overlay test check instead of blindly trusting my own visual read of the image file is what actually caught that the map script itself was the remaining stale piece left in the folder. I now check outputs against an independent overlay instead of trusting a silent script.

So I still have some massive outstanding gaps to handle. The main issue is that this map is just a single day of year snapshot at DOY 273 rather than a full continuous seasonal time lapse animation across the months. It is a limited data frame. More importantly, the low SIF drought zone I am seeing on my screen has still not been cross checked against any actual ground level drought impact reports from local district offices. I know that the visual shapes match what I would generally expect from known historical Marathwada drought geography patterns, but looking consistent with expectations is absolutely not the same thing as being fully confirmed by official field statistics. Cannot stretch my claims. So both of these missing steps go straight into my diary, not me treating this single map layer as more conclusive than it actually is.

Entry 6

I was just guessing before. I just took the common talk that Marathwada had droughts in 2015 and 2018, and that 2020 was a normal monsoon time, but I nvr checked the real numbers myself for this project. So, to fix this gap before writing the final report, I opened Earth Engine and downloaded CHIRPS daily rain data for the identical Marathwada area and the same June to December months that I already used for my SIF and NDVI work. Needed a real baseline. So, I also pulled 20 years of rain data from 2001 to 2020 (a long time climatology average) because comparing just these 3 years with each other makes no sense if I want to see what a real "normal" year looks like.

The results settled it. Found out that the 20-year average rainfall for this season is 826.4 mm (the standard deviation is 158.8 mm). Now look at the actual years. In 2015, the rain was just 648.9 mm, which means it was 21.5% below the normal average... Then 2018 was also bad with 674.8 mm, so it was 18.3% below normal. But 2020 was different and very wet, coming at 1066.6 mm, which is 29.1% higher than normal. So, the drought and normal tags I used before are not just wild assumptions anymore because my own data clearly proves the rain shortage. Both 2015 and 2018 had a massive lack of rain, and 2020 was just a proper heavy monsoon year.

Then I saw a weird thing. Put these new rainfall figures right next to my old SIF NDVI lag calculations, and something quite unexpected came up (and this part does not fit the simple story I was hoping for, but here it is). See, the rain gap in 2015 was -21.5% and in 2018 it was -18.3%, which means they are almost same same with just a tiny 3% difference between them. But my data for the average SIF to NDVI lag shows a massive 24.2 days gap for 2015, while 2018 had a lag of only 5.1 days. They are miles apart. If less rain was the main reason for a bigger lag, then these 2 dry years should show a similar lag time, but they do not. So, whatever is making the lag act so strangely between these 2 drought times, I cannot just blame it on how much total monsoon rain went missing.

So I am treating this as a limitation of my work. Maybe the main reason is about when the rain actually failed (like, missing rain in early June versus late September can change how SIF and NDVI talk to each other, even if the total seasonal water loss looks identical), or maybe the soil was already too dry from the previous year, or maybe it is some other factor my current dataset misses. I just do not have the right data in this project to test these ideas.

But this extra rain check helps me. Now, the whole "drought year" setup I am using is not just some copy pasted hearsay from local news, because I tracked and calculated it myself using the same code setup. Also, my old point that "drought makes the SIF NDVI lag bigger" is wrong still stands on a much stronger ground. Now that I have the actual CHIRPS rain numbers with me, I know for sure that none of my selected "drought years" had good rain after all.

Entry 7

So I already verified the district rain numbers. But then I decided I needed a proper map view for this rain data, just like the spatial maps I previously coded for my SIF data. Also I did not want just a boring table with 1 single number printed for each district (that tells you nothing about the actual geography). I wanted a proper map so I could directly see and match the exact spots where the rain failed against the specific areas where the SIF crop stress was popping up.

So I coded this map using 3 separate steps. I wanted it to look exactly like my old SIF maps, so 1st I made an interactive Folium choropleth map. For this, I gave colors to each of the 8 districts based on their rain anomaly

percentages, and I added different toggles for all 3 years so a user can hover or click them on their internet browser. After that, I used GeoPandas and matplotlib to make a fixed, static version. This one has 3 maps sitting side by side (one for each year) and I typed the exact rain anomaly numbers directly inside each district boundary because this format works much better for the paper.

These 2 maps showed a very clear point. Back in 2015, the west and south districts like Aurangabad, Bid, Jalna, Latur, and especially Osmanabad had very low rain compared to normal, but Hingoli and Nanded in the east side were almost okay. Then in 2018, the identical west to east difference happened again (but this time it was even more extreme). Aurangabad and Bid both dropped past a huge 35% rain deficit, while Hingoli and Nanded actually got extra rain that same year. This is an important point. My old total average number for 2018 said the whole region had an 18% rain shortage, which is technically correct on paper, but it hides the real truth that the drought was mostly hitting just the west and south areas while the east side was close to normal. Calling the whole place a "drought year" is a bad generalization because the actual ground reality was really uneven.

Made the 3rd map. This was my main plan from day one because I wanted a single joint picture where SIF maps sit right on the top row and the rain anomaly maps sit directly underneath them for all 3 years. Doing this side by side lets me look at both patterns together. When I put them side by side, the matching spots became super clear. That deep red color showing low SIF in the southwest part during 2015 and 2018 is sitting exactly over the deep red rain deficit zones on those same years. Also, the matching places that turned nice and green because of extra rain in 2020 are the same ones showing very high, green SIF health.

Take Nanded district as an example. Its rain barely dropped during both drought times, and guess what? My data shows Nanded stayed way greener in SIF compared to the other struggling districts in both 2015 and 2018. This gives me a big relief because the science actually makes total sense now. I am looking at 2 entirely separate satellite things (one is a faint plant light signal and the other is a rain estimation product from entirely different space sensors) and yet they are pointing at the same ground spots for the very same reasons.

Then my code broke a little bit. When I generated that big combined map, I saw a annoying layout bug where the main heading and the 6 small map titles were overlapping and crashing into each other. I solved this layout mess by shrinking the map sizes a bit to give the top text some extra breathing space (plus I added manual title padding to every single subplot box) because just shifting the main top title higher was not working properly. They needed a clean gap.

Here's what this actually does and does not prove. Just because 2 maps look similar to your eyes does not mean it is a real math test. So I did not calculate any proper correlation or regression equations between my district rain shortages and the SIF crop values across these 3 years... Doing that would be very bad science anyway (So I only have 24 total data pairs from 8 districts over 3 years, and they are not even independent data points because neighbor districts during the same monsoon share identical weather). So, my maps are just a simple, logic based visual match check, and I will write it down exactly like that.

Entry 8

My Entry 7 ended with a proper limitation. Had that nice looking dual map of SIF and rain, but I openly said it was just a visual match because I had not done any real math test on them. But I did not want to leave this gap empty before finishing the whole project. I went back and ran the actual math.

Took the files sitting in my `data/processed/` folder—specifically `sif_by_district.csv` and `rainfall_anomaly_by_district.csv`—and combined them using the year and district names. This gave me 24 rows of data points (which is just 8 districts multiplied by 3 study years). Rather than relying on just 1 simple test, I ran a Pearson correlation to check for a straight linear line, and then I also ran a Spearman rank correlation just in case the link was non linear.

The numbers came back higher than I expected. My Pearson r value was 0.837 (with $p < 0.001$) and my Spearman rho value was 0.857 (also with $p < 0.001$). Both tests say the same thing. This is not a confusing or borderline result hanging near the passing mark. It is a clear positive connection showing that when rain drops, SIF values also drop across these districts and years.

But the identical warning about data independence from my Entry 7 applies here (just that I can explain it better now). See, 24 data rows looks like a good, big sample on paper, but the 8 districts inside a single year are not independent from each other because they touch geographically and share the same regional monsoon clouds. So, the true number of separate events here is actually closer to 3 (just 1 per year) rather than 24. I will write down the correlation score as the code gave it, but the effective sample is far smaller than 24.

So what changes now? My simple "visual check" idea from Entry 7 now has a real, solid number behind it to back it up. But it changes absolutely nothing about my core result from Entries 3, 4, and 6 (which proved that a worse drought does not simply increase the SIF NDVI lag time). This new correlation is just showing that SIF and rain values match nicely across different places on the map, which is a separate topic from the time lag question. I do not want to mix up these 2 different ideas just because both of them happen to prove the main point of my project—that SIF and rain are tracking real, physical connections on the ground.

Entry 9

I found a big mistake. While reading my own paper's Physical Basis part again, I saw a fact about GOSIF that I wrote down but nvr actually fully thought through. Had written that GOSIF is not a direct scan from a single satellite, but instead, it is a mixed up statistical model that takes sparse OCO-2 SIF points and glues them together using continuous MODIS light reflection data and weather reanalysis models. This is 100% correct because that is how they build the dataset. But I missed writing down the actual meaning of this for my project (and this is a major catch): the identical MODIS reflectance data that GOSIF uses to fill in the blank days between OCO-2 satellite passes is also the main raw data for the NDVI curve that I am testing it against!

Basically, this means my SIF and NDVI are not 100% separate measurements in this study. On the exact days and specific fields where the OCO-2 satellite missed taking a direct photo, the GOSIF estimate is actually guessing values using the same MODIS light pictures that give us our NDVI numbers. (So the 2 datasets are technically copying from each other's notes). So I am not saying my SIF before NDVI time lag answer is fully wrong, and I am not saying GOSIF is a poor dataset. Everyone uses it because raw OCO-2 data points are just too few and far between to cover a whole big region like Marathwada. But I need to say this clearly in my notes: a big chunk of matching data is already stuck inside both variables because of the data product I picked. Right now, I have zero time and zero extra files to calculate how much of my lag comes from real plant science versus this messy model mixing.

So I am writing this down right now as a straight limitation.

Entry 10

So I went back to my old writing. Next I reported the massive difference between the 2015 lag (24.2 days) and the 2018 lag (5.1 days) since Entry 3, but when I reread my draft today, I realized I just stated the fact without explaining it at all.

Now, looking at it properly, I know I cannot prove the exact reason with my current files. But I can guess some real logical reasons. Both years had almost identical rain shortage numbers, so if total water loss was the only driver, the lag days should look like twins, but they look very different instead. I can think of 3 possible reasons for this massive gap (even though I have zero data to test them right now)—maybe the rain failed early in June instead of late September, or maybe the big 16-day gap in NDVI satellite photos created a bad timing calculation error in 1 year, or maybe local farmers simply changed their crop types between these 2 seasons. Do not have any crop calendars or government sowing data to verify this mess, so I am just writing these down as 3 open guesses and not picking one blindly.

Entry 11

I checked everything again. While going through my raw data files and not just my final text paragraphs, I found a huge mistake (and here is what happened, same as the other code mistakes earlier in this log).

Here is what went wrong. Ever since my Entry 4 work, my `data/processed/clipped/` folder had 2 separate files sitting inside it for every single date. So I had the old rectangular crop file named `GOSIF_<year><doy>.tif` from my old broken script, and I also had the new, correct polygon mask file named `GOSIF_<year><doy>_clipped.tif` from my fixed script that cuts data using the actual Marathwada border lines. Keeping both files would be okay, but my aggregation script `aggregate_sif.py` was looking for the pattern `"GOSIF_*.tif"` using a regex filter `r"GOSIF_(\d{4})(\d{3})\.tif"`. See the problem here? This regex expects `.tif` right after the date numbers, so every single correct `_clipped.tif` file failed this rule and got skipped silently as a bad name while the old, leaky rectangle files matched it perfectly fine. So, my code was secretly building `marathwada_sif_timeseries.csv` using the wrong, unclipped data this entire time (even though I thought my Entry 4 and 5 fixes had updated everything).

Missed this in Entry 4. Back then, I wrote down yearly SIF averages of 0.137 for 2015, 0.151 for 2018, and 0.219 for 2020 as a quick proof check, thinking my border fix was working perfectly. Those numbers are actually right for the old box shaped files (it is just that I had no idea I was reading the wrong folder files). My check was just comparing the old bad data against itself under a different name in my head instead of comparing it to the new polygon cropped outputs because my script never read the new files!

But thankfully my district level math was still safe. The script `zonal_stats_sif.py` (which I used for making my map visuals and calculating the SIF rain correlation scores) was not broken at all because that code directly calls the exact `_clipped.tif` text strings in its file path rather than using a loose glob search pattern. So, my Pearson score of 0.837 and Spearman score of 0.857, plus all my district maps, were nvr wrong to begin with.

So I fixed the script. And I changed the file matching code and the regex text string inside `aggregate_sif.py` to target only `GOSIF_*_clipped.tif` and `r"GOSIF_(\d{4})(\d{3})_clipped\.tif"`. This means my code will now strictly load the real polygon cropped outputs from `clip_gosif.py`, even if some old files are still sitting inside that folder. Then, I re ran the full analysis to regenerate my data sheets like `marathwada_sif_timeseries.csv`, `marathwada_sif_ndvi_merged.csv`, `sif_ndvi_lag_by_threshold.csv`, and `final_summary_rainfall_lag.csv` using the proper data (and I also updated the 2 main charts `sif_vs_ndvi_seasonal_v2.png` and `sif_ndvi_lag_by_threshold.png`).

Did the core results flip? Not really. My new fixed lag calculations are 24.3 days for 2015, 6.9 days for 2018, and 28.7 days for 2020 (the old wrong numbers were 24.2, 5.1, and 25.5). The average lag during drought years changed from 14.6 to 15.6 days, while the normal year lag went up from 25.5 to 28.7 days. SIF still drops down before NDVI across all 15 threshold year combinations. My main point—that drought years actually have a shorter lag than normal years (rejecting H3)—still stands strong on the correct border data. But I should highlight that the 2018 lag number jumped the most from 5.1 to 6.9 days (that is roughly a 35% change), which is important since it is the smallest and most striking lag value in my whole project.

Need to clean up. I still have those old, unclipped rectangle files sitting inside my `data/processed/clipped/` folder right next to my correct data. Chose to leave them there for now instead of deleting them in a hurry because I wanted a record of what the bug had been reading. But I must delete them before finalizing this GitHub repository. (Their presence is the exact reason this bug stayed hidden for so long.). If anyone writes a new script later that uses a loose `GOSIF_*.tif` search pattern, it will break again the same way until those bad files are gone...

Entry 12

Used only one logic till now. Every single lag value I wrote down in this project, even after my Entry 11 fix, comes from the same threshold crossing method. But this technique has a fixed internal bias because it only tracks when the actual plant decline begins (especially when looking at high numbers like 90% or 80%). It cannot tell me if my main results will stay same if I ask the question in a different way. So, before closing this project for good, I decided I needed a solid 2nd opinion from a fresh approach.

I chose time lagged cross correlation. 1st, I took my SIF and NDVI numbers, changed their scales to 0-1, and focused only on the downward slope months after the peak growth. Then, I put them on a shared daily grid, subtracted the average values to de mean the lines, and checked which day shift gives the best matching correlation score. This test asks a unique question compared to the old threshold setup (rather than catching when the downfall starts, it finds the best single time shift to match the whole shape of both charts). If these 2 different math styles give the same story, then the result is much more robust.

I caught a mistake before it reached my results. In my 1st test run, I allowed my code to search for time lags all the way up to 60 days, but then I noticed that 2 of my 3 study years had their peak correlation scores sitting dangerously close to that 60-day limit (specifically, 2020 showed a peak at day 56 out of 60). This is a classic artifact in cross correlation math (if you test a lag that is almost as long as your entire data timeline, you have too few overlapping days left to get a trustworthy score, so the peak you see is just the search window running out of space rather than a real best match). To fix this, I capped the max allowed lag to just N/4 of my total downward slope period (which is a standard math rule, giving about 32 days here) and then I re ran the script. Now, the peak lag days for all 3 years sit safely inside the middle of my search range instead of getting stuck on the extreme edge, so this is a real best match spot and not just a code artifact of where I stopped looking.

The new results came out. I found the best cross correlation lag points at 13 days for 2015, 4 days for 2018, and 7 days for 2020 (all 3 are positive numbers, they show super high correlation values of 0.99, 0.99, and 0.99, and none of them got stuck on my search cutoff limit). Because every single year gives a positive lag day, my new separate method independently checks out and supports the main point at this stage: SIF definitely drops down before NDVI across all 3 years. This makes my H1 idea way stronger than before.

But not every part matched. My old threshold script ranked the lag sizes from biggest to smallest as 2020 (28.7 days), then 2015 (24.3 days), and lastly 2018 (6.9 days). But this new cross correlation script ranks them

as 2015 (13 days), then 2020 (7 days), and lastly 2018 (4 days). (Both methods agree that 2018 has the smallest lag of all.). But they fight each other on whether 2015 or 2020 has the absolute biggest lag, which is the exact base comparison for my whole H3 drought amplifies lag debate. The simple version: the basic idea that SIF drops before NDVI is now highly trusted because 2 different math styles show it, but the exact day counts and the ranking of the years are still fuzzy because 2 equally valid methods give different orders. I am putting both statements clearly in my final paper and inside this log.

My original threshold numbers do not change at all. I am not replacing my old calculations at all. This new test is just an extra bonus check (like looking at the same map from an entirely different angle) to give my project more depth.

Entry 13

So I sat down to critically review my whole project folder today. Once my paper drafts, project journal, and dev logs were all in a stable state, I went back through my code work with fresh eyes the exact way I would expect a tough reviewer to do, and not just lazily treating "it runs and the numbers look right" as the final finish line. I wanted to find my own mistakes 1st. Going back through my folders turned up the identical list of gaps every single time. I found that I had zero uncertainty quantification around my lag estimates, my district level correlation's non independence was just asserted blindly without a real measurement, my Google Earth Engine data acquisition step was still unscripted, and—the one that bothered me the absolute most—my RQ3 question had nvr actually been answered anywhere in the final paper draft. It was an embarrassing list of omissions. This log entry records exactly what I did to solve each of those holes, and it also notes down the messy code logic that I tried and abandoned along the way before getting things right.

My 1st bootstrap attempt failed. Initially tried running a moving block bootstrap loop directly on my interpolated daily SIF and NDVI time series datasets by chopping each year's seasonal decline curve into overlapping 10-day chunks. My plan was simple. I chose to resample those blocks with replacement, recompute the cross correlation lag on the shuffled data frames, and repeat the whole thing 2,000 times on my machine. Every single confidence interval collapsed down to a flat [0.0, 0.0] marker. This was not a tight range at all, but just a broken degenerate artifact in my output terminal. It was an actively misleading result. If I had reported this number blindly as the lag is exactly zero days plus or minus nothing, I would be reporting an artifact as a result.

So I looked into why it broke. The real reason is that these seasonal decline curves are strongly non stationary because they trend monotonically downward across the full monsoon harvest months. Shuffling chunks of a trending line really destroys the pattern itself. My resampled time series stopped looking like a real plant decline curve at all, so my cross correlation script just grabbed whatever was left over in the array. The loop was just picking up random noise centered near zero lag. I realized I was just measuring what happens when you erase the crop signal, instead of sampling the real uncertainty inside that signal. Had to change my approach immediately.

So I used case resampling instead. I stopped shuffling random data points on my interpolated daily grid, and chose instead to resample exactly which of my original 17 sparse 8-day satellite observations went into building that target year's curve with full replacement, then re interpolated and re ran the identical cross correlation script from Entry 12. This step fixed the math nicely. This change preserves the actual shape of the seasonal decline curve in every bootstrap replicate while still capturing actual physical uncertainty, and it mimics real life — specifically, it tracks the uncertainty of not knowing in advance which cloud affected

overpass dates you will actually get from the satellite during a messy monsoon season. My new results make perfect sense.

So I checked the new ranges on my screen. Found 2015 gave 13 days with a 95% CI of 5 to 20, 2018 sat at 4 days with a 0 to 9 range, and 2020 hit 7 days with a wide 0 to 32 boundary. The big catch is that every pairwise comparison between years overlaps on the timeline, so my exact ranking discussion from Entry 12 is not something my data can support with super high confidence. But my core direction remains safe. My lag calculations stayed non negative in 100% of my bootstrap replicates for every single year, and it was strictly positive in 84.5% to 99.9% of them. This is a double sided finding: a real quantified strengthening of my H1 hypothesis, but also a real quantified weakening of my confidence in the specific day numbers. Both facts are true at the same time. I wrote them down exactly like this in my paper draft rather than keeping a single point estimate that looked way more precise than it actually was on the grid. That's the review pass done.

So I ran a spatial test today. I wanted to quantify how much my district level correlation numbers lack independence by using a formal Moran's I metric script, since my old GA_Research_Paper.md draft always stated that the 24 district year observations backing my r = 0.837 correlation are not fully independent because the 8 districts sit right next to each other on the map. This was a basic blind assertion. I chose to fix this by building a proper Queen contiguity spatial weights matrix directly from the actual FAO GAUL district polygon boundaries with a calculated mean of 3.25 neighbors per district. Then I ran my Moran's I check using a heavy 9,999-permutation significance test for both my mean SIF data and rainfall anomaly tracks separately for each calendar year. My script outputs gave me a massive reality check.

Every single check came back significant. All 6 year by variable combinations returned a p value below 0.05 with the actual I value ranging cleanly from 0.26 to 0.55 against an expected value of −0.14 under spatial randomness. Numbers hold up fine. This does not change my final correlation coefficient number or invalidate my project data because the underlying spatial pattern is real and cross validated against my SIF charts in Section 4.3. But it changes how I count my rows. This means I can now state precisely that my effective sample size is actually closer to a tiny 3 instead of a big 24, rather than just waving my hands at the issue. This fixes my statistical claim.

So I scripted the Earth Engine data download step today. One caveat about what this scripting step actually means: the NDVI, cropland mask, and CHIRPS rainfall extractions were originally all run by hand inside Earth Engine's browser Code Editor right from the start of this project, and every single version of my Limitations section has explicitly said so. Wanted to automate this messy workflow. Wrote a fresh python file called src/acquisition/gee_data_acquisition.py using the earthengine api package and geemap module. My new code reconstructs exactly what Section 3.1 of my paper describes by setting up SummaryQA filtered MOD13Q1 bands, the MCD12Q1 cropland mask with classes 12 and 14, CHIRPS seasonal rainfall totals, and the 20-year climatology baseline all running against my official FAO GAUL Marathwada district polygon. This is a clean script layout.

But I want to be super precise about what this file does and does not fix. I do not have an authenticated Google Earth Engine account on the machine I was working from right now, so I have not executed this script and cannot confirm with my own eyes that it reproduces my original interactive browser output exactly. It is an unrun backup file. What it actually does is turn a purely prose described, only mentally reproducible manual step into a checked in, version controlled repository artifact that anyone with real Earth Engine API access can easily run and inspect. Chose to write this exact distinction explicitly into my paper's Limitations text box rather than letting the mere presence of this python file imply more than it has actually earned. Repo's updated with the new file.

I finally answered RQ3 today. This is the part of my original notes I feel worst about missing, because RQ3 and the research questions list in GA_Project_Report.md always framed my policy motivation partly using the official drought declaration timeline rather than just a simple rainfall deficit score. But my section 4 inside GA_Research_Paper.md forgot to run this check. My older section 3.5 only looked at a rainfall deficit comparison which answered a real question but missed the government timeline angle. Had to fix this gaping hole. I ran a web search to find a verifiable official Maharashtra drought declaration date for each study year in my folder. For 2018 this pass was super straightforward and well documented, since the state government officially declared a massive drought in 151 talukas across 26 districts including all 8 of my Marathwada study districts on 31 October 2018, based on matching Zee News and EPW news articles from that week. This gave me an exact match.

But my 2015 data search failed. Did not find a comparably clean single date for 2015 because media coverage of the 2015 Marathwada drought is heavily dominated by the subsequent 2016 Latur water crisis reporting rather than a properly dated 2015 Kharif declaration paper. It was a messy archive hunt. Also I chose to report this timeline comparison for 2018 only and stated this data limitation explicitly in my text rather than picking some plausible sounding date that I couldn't actually verify with a clean link source. At least the timeline comparison isn't overstating things anymore.

I looked at the actual 2018 numbers. My data shows SIF crossed its 90%-decline threshold on 8 September 2018, NDVI crossed it on 12 September 2018, and the official state declaration came out way later on 31 October 2018, which means a massive delay of 53 and 49 days respectively. Sitting with these numbers changed my mind. Realized that the SIF vs NDVI time gap which is the whole point of my paper is just 3 to 4 days on the grid. But the satellite to government gap is roughly 7 full weeks. These 2 scales are really different. This RQ3 comparison needs me to say this clearly in my paper: the bigger finding is that any satellite signal beats the current 7-week plus institutional process by a wide margin. SIF is just a neat extra way to optimize that big jump. I would rather state my policy case at the size it has earned.

My core baseline numbers didn't change. Every single update I threw into this entry is either a precision improvement like bootstrap CI or Moran's I metrics, or a truly new comparison like RQ3 that I definitely should have run right from the absolute start of my project. I am just tuning things up, and my claims are now written down with much more appropriate and modest confidence levels than my older drafts showed. That rounds off the research setup for now.

Entry 14

My paper's Limitations section looked no different across every single version. I kept repeating that 3 years consisting of 2 droughts and 1 normal season was way too small a sample size and that my 2 drought years differed substantially from each other. So I got tired of just talking about this gap, and decided to actually fix this issue and not just lazily stating it as a permanent limitation block in my text. Added 5 more years to my folder. Grabbed data for 2016, 2017, 2019, 2022, and 2023. This big expansion step brings my entire satellite study to 8 years total across my timeline sheet. It makes the data much stronger.

1st step: getting the raw data.

I checked the database availability lines 1st. I found that GOSIF v2's 8-day product covers everything from 2000 to 2024, so all 5 of my new target years exist cleanly in their database. I sat down to write the acquisition files. I wrote a new script called src/acquisition/download_gosif_new_years.py to pull them in using the same DOY-153-to-361 seasonal window I already used for 2015, 2018, and 2020. So I kept the identical file naming convention... This choice means my old clip_gosif.py script requires zero code edits to pick up and crop the

fresh files automatically. Then I updated my Google Earth Engine pipeline. I extended the STUDY_YEARS list array inside src/acquisition/gee_data_acquisition.py to all 8 years. Then I also added 2 brand new CSV export code blocks, choosing to download region mean rainfall and by district rainfall tables directly as proper CSV files because my original script only printed raw rainfall totals straight to the terminal console. That old style was fine for typing 3 numbers by hand but it fails badly when you have 8 years to track. Download setup is ready to go.

Hit a big surprise while setting this up. Cannot reach the main GOSIF data server directly from the network I was working on because it throws a nasty 403 Forbidden code error on every single request, browser user agent header or not. It is a complete network lockout. This is the same kind of nasty IP range block that stopped Sentinel Hub when I was working on my DOUBLE_JEOPARDY project folders. So I had to alter my system plan — the script download has to run directly on my own home machine hardware, which is the same way that other project's satellite data pulls did. It is an irritating extra step.

I want to be upfront about what is not yet done:

Neither script has actually been executed yet. My raw GOSIF download needs to happen from a normal home internet router line, and my GEE extraction code requires my authenticated personal Earth Engine account. And I have not triggered the code loops, since this is the identical login requirement the original unrun acquisition script already carried in its file headers. My file metrics are still old. Every single number inside this project workspace as of this log entry is still the original 3-year result from my 1st pass. This entry just marks the absolute beginning of my expansion work. The very next entry will cover what actually came back once both scripts finish running and I re process the full analysis pipeline against 8 years instead of 3. I need to run my clipping, lag analysis, cross correlation loops, bootstrap confidence intervals, and Moran's I scripts again. This will update every single downstream document in my folder. Picking this back up tomorrow.

Entry 15

I ran download_gosif_new_years.py 1st. All 135 files representing 5 years multiplied by 27 composites came down clean on my very 1st attempt because this ran directly from my own internet connection as expected. I did not face any old GOSIF blocking bugs here. My code script worked perfectly. But gee_data_acquisition.py was a whole different story. Every single one of the next 3 problems I hit turned out to be an actual bug inside code blocks that had nvr actually been executed before this pass. So I was testing fresh code path logic here.

Bug 1 — no registered Cloud project.

So I hit a major initialization block right away. Modern Earth Engine now strictly requires you to type ee.Initialize(project=...) using a registered Google Cloud project ID string inside your setup. My script only carried a bare ee.Initialize() line and a commented out ee.Authenticate() command, since both lines were dusty leftovers from old days before Google added this strict cloud project requirement to their API. I had to fix this setup line. I added both parameters properly using my own personal EE project ID string.

Bug 2 — MODIS clip against the wrong projection.

My next task failed with a bad error message. My green NDVI extraction crashed with a nasty projection transform error while trying to clip a MOD13Q1 image sitting on its native sinusoidal grid SR ORG:6974 against my Marathwada boundary polygon shape in EPSG:4326. It was a total mess. Some specific district boundary geometries just do not transform cleanly onto that curved raster edge line. I looked into the code functions. Found that the .clip() call was redundant anyway, since the reduceRegion() function call right after it

already restricts the mean calculation loop to the same geometry limits. I deleted that useless clipping line from my script file. Runs clean now.

Hit Bug 3 next. I noticed a whole district went silently missing from my spreadsheets — even though the code finished running with zero terminal errors, I did not trust it blindly and actually counted the rows in my final output table. I am glad I checked. My by district rainfall export file listed only 56 total rows, not the 64 rows representing 8 districts multiplied by 8 years that it should have produced. I found a bad naming mistake inside my code setup. My MARATHWADA_DISTRICTS list used the standard common English spelling "Beed", but the old FAO GAUL 2015 dataset stores this specific area under the name "Bid" inside its ADM2_NAME column field. This caused a massive silent leak.

The filter line did not help. The ee.Filter.inList() command never throws a crash error when you type a spelling that matches absolutely nothing on the grid. It just drops the data quietly. My code silently dropped that single district from my merged region boundary layer and deleted it from every single district level export across all 8 tracking years. It messed up my whole boundary shapefile logic. I chose to double check the real text. Next I verified the proper spelling string against my raw district GeoJSON layer sitting inside my data/raw/ folder and fixed my python list text right away. This solved the problem. So I re ran my entire extraction script chain a 2nd time on my machine. Then I opened up my final output CSV file and manually counted every line to make sure: 8 districts multiplied by 8 years gave me exactly 64 rows with all 8 names present on the sheet. Every downstream analysis code now uses this fully corrected table file.

So I ran my rainfall code next. I triggered rainfall_analysis.py against my corrected 8-year regional rainfall totals, and it gave me the most interesting result of this whole expansion pass that I absolutely had not expected going in. This output really surprised me. I used the same drought threshold implied by my original study, which means roughly half a climatological standard deviation below the 2001–2020 mean value. My data showed that 2015 at −21.5% with z = −1.12 and 2018 at −18.3% with z = −0.95 remain the 2 absolute driest years, exactly like my original 3-year check stated. The other years looked very safe. None of my 5 newly added years even come close to that dry line. My calculations showed 2016 hit +9.5%, 2017 got −1.1%, 2019 reached +13.3%, 2022 jumped to +36.8%, and 2023 sat at −3.7%, keeping them all within roughly 1 standard deviation of a normal season. This is a massive shift. So across my full 8-year record, only 2 out of 8 years actually qualify as real drought years by my own project definition. My original 3-year sample carried a 67% drought rate by pure chance of which years I picked 1st, which was way more drought heavy than Marathwada's real climate history. Will write this down in my limitations section. It does not break my drought year findings at all, but it proves the original balance was unrepresentative of reality. My 8-year timeline gives a much clearer picture of what's actually normal here.

I changed my hardcoded variables today. I switched every hardcoded DROUGHT_YEARS = {2015, 2018} literal string across my lag, cross correlation, and bootstrap scripts to dynamically derive that set straight from the actual rainfall anomaly file output instead of typing it by hand. It still resolves to the identical 2 years right now. But it means my classification is computed dynamically from the data going forward rather than being typed out manually, which matters way more with 8 full years in play than it did with just 3. This builds a much better pipeline structure.

Still pending as of this entry:

Still have some massive tasks left on my queue. My clip_gosif.py and aggregate_sif.py scripts—the 2 pieces that turn the raw GOSIF rasters into the actual working SIF time series data frames—need to run fully against all 216 raw files representing 27 composites multiplied by 8 years. This processing task has to happen locally on my own machine. Not moving them anywhere else, because transferring 11+ gigabytes of raw heavy

rasters over the network isn't worth moving anywhere. The clipped output size per file is actually under 40 kilobytes, so only that tiny cropped text output needs to travel anywhere else for my dashboard or plots. Once that long calculation pass is finished, all my SIF side findings will get reprocessed. I need to re run my lag analysis, cross correlation loops, bootstrap confidence intervals, Moran's I metrics, and district level correlations against the full 8 years. My folder is stuck with the original 3-year numbers as of this log entry. I have a long night of data processing ahead.

Entry 16

I ran my main SIF processing tools. Executed clip_gosif.py and zonal_stats_sif.py directly against all 216 raw GOSIF image files on my own machine, matching how I handled my download scripts back in Entry 15. Both files are fully generic. I used standard glob based file discovery strings with zero hardcoded year lists typed anywhere inside either script, so neither file required a single code edit to detect the 5 new data years. This design choice saved me. I deliberately wrote them this flexible way back when I was only tracking 3 years, and it paid off exactly as intended here during this heavy expansion phase.

My output tables came back looking fully populated. The marathwada_sif_timeseries.csv file populated with exactly 216 data rows representing 27 composites multiplied by 8 full years, and sif_by_district.csv generated 64 rows representing 8 districts multiplied by 8 years. These grid sizes matched my region level rainfall rows from Entry 15 down to the line. This served as an excellent manual sanity check that my loop hadn't quietly dropped a year or skipped a district anywhere on the SIF processing side. Dataset's complete now.

I got the real 8-year SIF data in hand now. Reprocessed every single downstream analysis script against this big dataset, running compare_sif_ndvi.py, lag_analysis.py, cross_correlation_lag.py, bootstrap_lag_uncertainty.py, sif_rainfall_correlation.py, spatial_autocorrelation.py, and rq3_official_declaration_comparison.py on my terminal machine. It took a lot of processing hours... Also I changed my code setup to be more smart. I switched every remaining hardcoded DROUGHT_YEARS = {2015, 2018} text string straight to the dynamic, rainfall anomaly derived classification version that I already started coding back in Entry 15. Now the math is calculated uniformly everywhere. So I fixed 2 tiny deprecation warnings that popped up on my screen while working inside those script files, replacing the old matplotlib.cm.get_cmap line with matplotlib.colormaps[...].resampled(...) and swapping out geopandas's old .unary_union function with the modern .union_all() call. This cleanup pass made my entire project backend stable.

Found a huge 2018 exception. This is by far the single most interesting thing that came out of my full data reprocessing pass. My scripts are telling a very weird story here. Every single lag estimation method—meaning threshold crossing calculations, cross correlation loops, and the extra bootstrap check on top—all agree that 2018 behaves differently from the other years. I calculated its mean threshold crossing lag and it came out marginally negative at −1.1 days, which stands out against the positive 4.2 to 23.8 days range for every other year in my sheet. It broke my pattern. Its cross correlation lag value is exactly zero. Also, only a tiny 8.1% of its bootstrap replicates show a positive lag score versus the high 42 to 97 percent seen for every other year in my study. So I checked it a 4th way using my RQ3 comparison against the official 2018 drought declaration paperwork. My data shows that NDVI actually crosses its 90%-decline threshold *before* SIF does during that specific year, which flips what my original 3 year study reported early on. I am stuck — I simply do not have any specific crop type records, local sowing date files, or any other covariate data rows that would explain why 2018 specifically behaves this way. So it just goes down as a plain, unexplained finding. I am not going to guess at a cause or average it into my drought year group. That's this entry wrapped up.

I found my other headline number today. Only 2 out of my full 8 timeline years are actually real, documented drought years on my spreadsheet. Already flagged this issue back in Entry 15 from the pure rainfall deficit side alone, and now it holds up identically once I bring my SIF dataset layers into the picture too. My original selection was highly skewed. My old 3 year sample being 2/3 drought years was, in hindsight, a considerably more drought heavy selection than Marathwada's actual 8 year historical climate record supports. Had a wrong picture of my home region early on.

Weaker, but still real, at the larger sample.

My district level SIF rainfall correlation dropped significantly. I calculated that the r value fell from my old r = 0.837 where n = 24 all the way down to r = 0.567 where n = 64 across the full sheets. My math is still highly significant. This is a solid, clear weakening of the signal strength rather than some tiny rounding difference, and that is how I report it. My Moran's I spatial calculations tell a similarly 2 sided story. I found that rainfall's spatial clustering stayed significant across all 8 working years, matching the 3-for-3 pattern I originally calculated. But my SIF results are mixed now — its spatial clustering is now significant in only 4 out of my 8 years on the map grid. It is real in some years but undetectable in others. This gives me a much more qualified, messy picture than my original all 3 years significant result implied. Logs are caught up now.

Had to fully rewrite my district level ranking section. My new 8 year calculations proved that Nanded, which used to be the highest SIF district in all 3 of my original baseline years, only leads in 2 out of 8 years now. Found that Aurangabad actually leads most of the time sitting at 4 out of 8 years, while 2 other districts lead once each. This is a different story. The consistently highest framing statement from my original 3 year study simply does not hold true at this much larger sample size. So, I chose to say so directly in my journal.

The bottom of my ranking chart looks way more stable. I found that Osmanabad stays stuck at the absolute lowest SIF rank in 7 out of my 8 working years. It is a highly consistent hotspot. But Bid takes over that lowest spot specifically in the year 2018. I checked why this happened. This is the same district that recorded the single lowest SIF value of my entire study, and it also matches the year's worst recorded rainfall deficit. My old timeline claims broke down too, since my original claim that every single year's lag is strictly positive is no longer true at 8 years because 4 years now tie at exactly zero days. This is a massive shift.

This log entry finally closes out my full timeline sample size expansion task. Every single number inside my project folder is now the real, updated 8 year result, not a messy partially updated mix of old and new data lines. Spreadsheets finally match up across the board.

Entry 17

Found a real bug today, and it's not a small one. Was doing another pass over cross_correlation_lag.py and bootstrap_lag_uncertainty.py, just reading the code slowly line by line and not trusting that it already worked. Both scripts search for the lag that best lines up SIF and NDVI, but the search range was written as `np.arange(0, max_lag + 1)` in both files. The search only ever checked lags from zero upward, meaning it was mathematically impossible for either script to ever come back with NDVI leading SIF, no matter what the actual satellite data showed. It's not that the data said "never negative." It's that I never let the code look there.

This changes what "SIF's lag is never negative" actually means. Every earlier entry in this log, and the paper itself up till now, reported the cross correlation lag as never negative and the bootstrap as showing 100% of replicates non negative in every year. Both of those statements were technically true, but only because I'd built

a search space that ruled out the other answer before it ever looked at a single number. That's not a finding; the code could not have returned anything else.

Fixed it properly rather than just patching numbers. In cross_correlation_lag.py, changed `lags = np.arange(0, max_lag + 1)` to `lags = np.arange(-max_lag, max_lag + 1)`, and added a small `lagged_corr()` helper that handles positive lag, negative lag, and zero lag as 3 separate cases since numpy slicing with a negative shift needed to go the other direction. Did the same 2 sided fix inside bootstrap_lag_uncertainty.py's `lag_from_daily()` function, and updated its boundary safety check (`abs(lag) >= len(sif_c) - 4`) so it protects against running out of overlapping points on the negative side too, not just the positive side like before. Reran both scripts from scratch on the real merged SIF/NDVI data.

The corrected numbers tell a whole different story, not just a wording fix. Cross correlation lag by year now reads: 2015 at 4 days, 2016 at 2 days, 2017 at 27 days, 2018 at −4 days, 2019 at 18 days, 2020 at 0 days, 2022 at −9 days, and 2023 at −10 days. Under the old 1 sided search, 2018, 2020, 2022, and 2023 all landed at exactly zero because zero was the best the search was allowed to find — now 3 of those 4 years, 2018, 2022, and 2023, come out clearly negative, meaning NDVI actually leads SIF in those years by this method. Only 2020 stays a true tie at zero. So instead of "SIF leads in half the years, ties in the other half," it's now "SIF leads in 4 years, ties in 1, and NDVI leads in 3."

The bootstrap confirms this isn't noise. 2022 and 2023 now show 99.4% and 99.1% of their replicates landing below zero — that's a strong, well resolved signal in the NDVI leads direction, not a coin flip. 2018 leans the same way at 87.4% of replicates negative, though its confidence interval still straddles zero so it's less settled than 2022/2023. Redid the pairwise year comparison too. Of the 28 possible year pairs, 24 still overlap and aren't statistically distinguishable, but 4 pairs now clearly don't overlap — 2015 vs 2022, 2015 vs 2023, 2017 vs 2022, and 2017 vs 2023. So the 2 clearest SIF leads years are now statistically distinguishable from the 2 clearest NDVI leads years. That's a real result the old 1 sided search could never have produced, since it wasn't allowed to put any year on the NDVI side to begin with.

Went through the whole project after this and fixed every place that repeated the old "never negative" framing. Updated GA_Research_Paper.md's methodology, results, abstract, discussion, limitations, and conclusion sections, regenerated both figures — cross_correlation_lag.png and cross_correlation_lag_bootstrap_ci.png — updated the 2 dashboard pages that show these numbers, GA_Executive_Summary.md, and README.md. Disclosed the bug directly everywhere and not quietly swapping in new numbers, same as I did for the Beed/Bid boundary bug and the Earth Engine Cloud project bug earlier in this log. This one actually changes a real scientific finding, not just a data processing detail, so it gets fixed the same way, out in the open.

While I was in there I also found a small leftover from Entry 16 that I said I'd already fixed but hadn't — clip_gosif.py still had the old `boundary.geometry.unary_union` call instead of `union_all()`. I thought I swapped this everywhere back in Entry 16 but missed this one file. Fixed it now too, small thing, but it's going in the log rather than pretending Entry 16 was complete when it wasn't.

A wrong search range that quietly guarantees its own answer is the kind of mistake this log exists to record, so it is written up in full here.

Entry 18

The journal's desk rejection listed 5 reasons, and one of them I could not argue with: no field validation. Everything in this study, all 8 years of it, is satellite comparing itself to satellite (SIF vs NDVI) or satellite comparing itself to rainfall gauges (SIF vs CHIRPS). Not once did I check any of it against what actually happened on the ground to an actual crop. I don't have the budget or the time to run a farm survey across 8 districts and 8 years, so I went looking for what the next best thing actually is.

Found it at the Ministry of Agriculture & Farmers Welfare — they publish district level, season level, crop level Area/Production/Yield numbers going back to 1997-98, every single year, free, no farm visit needed cause the government already did that collection work and published it. Getting the actual file was more of a headache than I expected though. First site I found, dataful.in, wanted ₹1499 just to download the CSV, which I wasn't expecting from what looked like an open data page. Found the same dataset for free on the India Data Portal instead (indiadataportal.com), signed in with just my email, no payment, and got the real file — crop-wise-area-production-yield.csv, 455,359 rows, all of India, 1997-98 to 2022-23.

Wrote src/analysis/crop_yield_validation.py to filter it down to just my 8 Marathwada districts and Kharif season, which is the monsoon crop cycle my SIF/NDVI trajectories actually track. Had to build a name mapping first cause the government renamed 2 of my districts in 2023 — Aurangabad shows up as "Chhatrapati Sambhajinagar" in this dataset and Osmanabad as "Dharashiv," even going back through the old years, so I mapped both back to the names I use everywhere else in this project.

For the actual check, I did the same z-score anomaly math I already use for my CHIRPS rainfall baseline (Section 3.5), just applied to yield instead — for every crop grown in a district with at least 10 of the 26 years present, scored each year against that crop's own 26 year mean and std, then averaged all the crop scores in a district-year into one Yield Anomaly Index, then averaged the 8 districts into one Marathwada number per year. 16 crops made the cut, mostly the ones I already knew were Marathwada's real Kharif crops — cotton, soyabean, jowar, tur.

Results actually came out solid. As a sanity check the yield anomaly correlates with my own rainfall anomaly at $r = 0.935$, $p = 0.002$ — tells me the method itself is measuring something real and not just noise. Against mean SIF it's $r = 0.669$, $p = 0.10$, right direction but not significant at only $n = 7$ years (2023 doesn't have a yield number published yet so it drops out). The part that matters most — my 2 drought years, 2015 and 2018, average a yield anomaly of −1.08, against +0.11 for the 5 normal years I have yield data for. Actual farmers, actual harvests, came out worse in the years I already call drought years from rainfall and satellite data alone. Ran a Welch's t-test on that ($t = -2.87$, $p = 0.13$) — not significant, but that's exactly what I'd expect outta only 2 drought years, same small-n problem that already runs through this whole paper's H3 framing. I report it as a consistent signal in the expected direction, not as proof.

This is not a field survey. It's yield statistics, not a farm-level stress survey, and it doesn't say anything about SIF's within-season timing claims specifically since yield is just 1 end-of-season number. But it is a genuine, external, government-sourced check that the years and region I'm calling "stressed" from satellite data alone were actually years Marathwada grew less food. Added it as new Section 4.11, updated the Abstract, Limitations, and Conclusion to reflect it honestly — supports the drought classification directionally, doesn't reach significance, and the paper says so.

Entry 19

Went after 2 more things off the leftover limitations list today — 2021 (the 1 year my own 8-year expansion never picked up), and the single pooled region-wide rainfall baseline that Section 6 already flagged as a gap. Both needed Earth Engine, so I pulled NDVI and rainfall for 2021 and a per-district climatology straight from

the EE Code Editor in my browser this time instead of the local Python script, since it's faster when I just need a couple exports and don't want to deal with local package installs.

2021 first. Wanted to actually check why I skipped it instead of just leaving that unexplained in my own paper. Turns out 2021 had 969.95 mm of seasonal rainfall, z = +0.90 against my usual 2001-2020 baseline — a wetter than normal year, not a drought year. So I wasn't quietly dropping an inconvenient drought season, good to confirm that directly instead of leaving readers to wonder. Still can't add 2021 as a full 9th study year though, cause I don't have its SIF data — GOSIF isn't on Earth Engine at all, that one still needs me to go download the raw GeoTIFFs by hand from the UNH data page and run them through clip_gosif.py same as every other year, and I haven't done that part yet. NDVI and rainfall for 2021 are sitting in data/raw/ now, ready for whenever I get to it.

Per-district climatology was the more interesting 1. Pulled each district's own 2001-2020 mean and std instead of the 1 pooled number I'd been using for every district alike. Turns out the districts really aren't that similar — Osmanabad averages 704.9 mm and Nanded averages 1009.0 mm over the same 20 years, both compared against a pooled baseline of 826.4 mm this whole time. So a district that's just normally wetter or drier than the region average was getting judged against a baseline that wasn't really its own normal.

Wrote district_climatology_anomaly.py to check how much this actually matters — recomputed every district-year's z-score against its own district baseline instead of the pooled 1, and compared. 6 out of my 64 district-year observations flip their drought/normal call depending on which baseline gets used — Nanded and Hingoli in 2015, Aurangabad in 2017, and Aurangabad, Bid, and Osmanabad in 2023. The 2 series still correlate strong overall at r = 0.840, so it's mostly a rescaling thing not a total reordering, but 6 real flipped classifications isn't nothing either. Checked whether any of this touches my actual drought-year definition (Section 3.5, which is a single region-wide number) — it doesn't, 2015 and 2018 stay the drought years regardless. But it's a real, honest limitation of the district-level correlation and Moran's I stuff in Sections 4.5 and 4.8, and now I've got the actual number instead of just a vague caveat sentence. Updated Section 6's limitations bullet and the Section 3.1 dataset intro to say all this directly.

Entry 20

Finally closed out 2021 today, the actual 9th study year, not just the rainfall/NDVI half I had sitting in data/raw/ since Entry 19. Downloading the raw GOSIF rasters myself turned into a whole thing — messed it up twice before getting it right. First attempt I grabbed the Monthly tier by mistake instead of the 8day tier, 7 files instead of the 27 8-day composites my whole threshold-crossing method actually needs. Fixed that, went back for the right tier, then somehow downloaded 19 files all stamped GOSIF_2002.tif.gz instead of 2021 — must've misread the year scrolling a long sorted folder listing, easy mistake when every filename looks the same shape. Deleted those, went back a 3rd time and Ctrl+F searched for "2021" specifically instead of scrolling, and that finally got me all 27 files, DOY 153 through 361, matching every other year in this project.

Processing them locally hit a real wall that had nothing to do with the science — ran out of disk space decompressing all 27 rasters at once (each one unzips to ~52MB, 27 of them is over a gig, on top of everything else already in the project folder). Had to rewrite clip_gosif.py's logic to go one file at a time instead — gunzip, clip to the Marathwada boundary, delete the decompressed raster, move to the next — so peak disk use stays under 100MB instead of 1.4GB. Same clipping math, just processed sequentially instead of all at once. All 27 clipped fine, 47.75% valid pixel fraction, matching the fraction every other year gets against the same boundary polygon.

Here's the part I had to be careful about — my working copy of this project only has the raw GOSIF files for a few of the 8 original years, not all 8, cause I never needed to move all of it off my own machine before. So I could not just rerun aggregate_sif.py and zonal_stats_sif.py normally like Entry 16 — that would've silently overwritten my real 8-year outputs with 2021-only data, since those scripts glob whatever's sitting in the folder and write straight over the output file. Instead I replicated their exact per-file math for just the 2021 rasters and appended the new rows onto the existing CSVs rather than regenerating them from scratch, checked every other year's rows were untouched before and after. Did the same append-not-overwrite move for the NDVI consolidated file and the rainfall-by-district file, using the anomaly math (mean=826.4mm, std=158.8mm, the same pooled baseline every other year in this study uses) against the 2021 district rainfall Entry 19 already pulled.

Once 2021 was properly merged into marathwada_sif_ndvi_merged.csv and sif_rainfall_district_merged.csv as a genuine 72nd/243rd set of rows, not a side file, I reran the full downstream chain against it — compare_sif_ndvi.py, lag_analysis.py, cross_correlation_lag.py, bootstrap_lag_uncertainty.py, spatial_autocorrelation.py, sensitivity_checks.py, final_summary.py, and crop_yield_validation.py (added 2021 to its STUDY_YEARS list too, government yield data actually goes up to the 2022-23 season so it's got a 2021 number). None of these scripts had hardcoded year lists left in them after Entry 15's cleanup, so every one of them just picked up the 9th year automatically off the merged files — exactly what that flexible-by-design choice back in Entry 16 was for.

2021 comes out clean and boring in the best way — every single check agrees it's a normal year, not a hidden drought I was missing. Mean threshold-crossing lag: 13.1 days, sitting right in the middle of my normal-year range, nowhere near 2018's −1.1 day outlier. Cross-correlation lag: +14 days, r=0.984, normal-shaped decline curve. Yield Anomaly Index: −0.13, close to 0, not the −1.08 average my 2 real drought years show. Rainfall z-score across every single 1 of the 8 districts came out positive, not just the region-wide +0.90 number Entry 19 already found. District-climatology flip count stayed at exactly 6, same as Entry 19, out of a now-larger 72 rows instead of 64 — 2021 didn't introduce a single new flip. Moran's I clustering for 2021 rainfall came out significant same as every other year, SIF clustering did not, matching the roughly-half-significant pattern the rest of the years already show.

So this is really n=9 now, not n=8 with an asterisk. Every table, every script output, every figure in outputs/figures/ regenerated off the real 9-year merged data. Still haven't touched the actual paper text yet — Section 3.1's dataset description, the abstract's "8 years" language, every count that says n=8 or n=64 across Sections 4.5-4.11, and the Limitations bullet about 2021 being unprocessed all need updating to reflect this before this goes back to JAG. Thts my next pass.

Entry 21

Went through GA_Research_Paper.md section by section today and fixed every single spot that still said "8 years", "n=64", or gave a stat computed off the old 8-year numbers — Highlights, Abstract, §3.1, §3.5, §3.9, §4.1 through §4.11, Discussion, Limitations, Conclusion, all of it. Every replacement number came straight off a fresh script rerun from Entry 20, not typed in from memory, so nothing here is guessed. Caught and fixed a few things along the way I would've missed on a quick pass: §4.1's "SIF declined ahead of NDVI in six of eight years" line was a qualitative call I couldn't reverse-engineer a clean numeric rule for, so I opened the regenerated seasonal trajectory figure and counted by eye instead — matches the §4.2 quantitative "8 of 9" finding, 2018 being the one visible exception. §4.7's bootstrap paragraph I'd first written as "SIF ahead in about two-thirds of the years," caught on my own re-read that this didn't match my own year-by-year classification, fixed it to the actual split — 4 years SIF ahead with real confidence, 2 years NDVI ahead with real

confidence, 3 years too close to call. And §4.8's Moran's I range had a typo, 0.21 where the real minimum (2019, 0.2577) rounds to 0.26.

Once the paper itself was done I realized the 4 companion docs all quote the same stats — GA_Highlights.md, GA_Executive_Summary.md, Policy_Brief.md, README.md — so a stale "8 years" sitting in any of those would leave the whole submission package internally inconsistent even with the main paper fixed. Went through all 4 and fixed everything grep could find: GA_Highlights.md's "7 of 8"/"3 of 8" counts and a missing crop-yield bullet that was in the paper but never made it into the highlights file, GA_Executive_Summary.md's whole findings table plus the year-by-year rundown, Policy_Brief.md's numbers throughout plus a crop-yield paragraph that had never been added even before 2021 (it still said crop yield "has not been checked" when §4.11 already existed), and README.md's Key Findings section plus 3 more stray spots elsewhere in the file (the 1-line project description up top, the sample-expansion sentence, and a comment inside the repo structure tree). Checked GA_Cover_Letter.md and GA_Author_Statement.md too while I was at it — cover letter had the same "8 growing seasons" line, fixed; author statement doesn't mention year counts at all, nothing to touch there.

So that's the actual answer to "which limitation is left to fix": the stale-numbers problem is closed out now, paper and all 5 companion docs match the real n=9 pipeline. What's left isn't fixable by adding more years or rerunning scripts — GOSIF being a modeled SIF product that shares some MODIS reflectance ancestry with the NDVI it's compared against is baked into the data source itself; 2018 stays an unexplained outlier across every method I've thrown at it; the crop-yield check in §4.11 will keep falling short of conventional significance as long as there are only 2 drought years to compare, more normal years don't fix a small-n problem on the drought side; and RQ3 stays a single-year comparison because 2018 is still the only year with a clean official declaration date to check against. Those go in the paper as honest, permanent limitations, not something to chase down further before resubmitting.

Entry 22

A full audit of the repo turned up a bug bigger than the search-range one in Entry 17. Every NDVI row in my 9-year file had a day-of-year one less than its own date column — 2015-06-10 labelled 160 when it's actually day 161. The cause is in gee_data_acquisition.py and the Code Editor .js file: `img.date().getRelative("day", "year")` counts from zero (1 January = 0), while GOSIF's filenames use normal 1-based day-of-year. My original 3-year NDVI export, done by hand in the Code Editor way back in Entry 2, had the right labels (161, 177...). The bug only came in when I rewrote the extraction as a Python script for the 8-year expansion.

Why one day mattered so much: compare_sif_ndvi.py matched each 8-day SIF date to the *nearest* 16-day NDVI date, and every other SIF date sits exactly 8 days from two NDVI composites (SIF 169 is 8 days from both NDVI 161 and 177). With the wrong label the tie always broke toward the later NDVI composite, whose window starts after the SIF composite has already ended. So half the SIF dates got NDVI from the next period, NDVI looked about 8 days earlier than it was, and every lag shrank by about 8 days.

Proof it's this and not anything else: I ran the current SIF data against the original 3-year NDVI file (correct labels) and got 24.3 / 6.9 / 28.7 days threshold-crossing and 13 / 4 / 7 days cross-correlation for 2015 / 2018 / 2020 — exactly Entry 11 and Entry 12. So the SIF side never changed. The drop in Entry 16 that I blamed on the Bid boundary fix was actually this date bug arriving in the same pass. That attribution was wrong, and it's now corrected in the paper.

Fixes: `.add(1)` on the day-of-year in both Earth Engine scripts; `doy` recomputed from the `date` column in all 11 NDVI CSVs; and compare_sif_ndvi.py now pairs each SIF composite with the NDVI composite whose 16-day window contains it (`direction="backward"`) instead of the nearest one, so there's no tie to break at all. sensitivity_checks.py's alternative is now interpolation at each composite's centre date. While rerunning I also gave the bootstrap its own random seed per year (sharing one stream meant adding 2021 had silently changed 2022 and 2023's bootstrap numbers), fixed the Moran's I seed (p-values were moving slightly between runs), fixed the RQ3 script's `%-d` date format that crashes on Windows, and relabelled its "90% of seasonal peak" events — a 0.9 threshold is 10% of the way down from the peak, not 90%.

Corrected results, rerun on my own machine: threshold-crossing lag 24.3, 31.8, 29.6, 6.9, 25.2, 28.6, 21.1, 13.1, 12.2 days (2015-2023) — SIF leads in all 9 years, drought mean 15.6 vs normal 23.1, so H3 still isn't supported. Cross-correlation 13, 11, 16, 4, 26, 7, 22, −1, −2 days; 2019's 26 is pinned to its ±26-day search edge. Bootstrap: only 2015 and 2016 have intervals that exclude zero, and no year supports NDVI leading (2022 is 52.5% negative, 2023 80.5%, both intervals include zero). 3 of 36 year-pairs don't overlap. RQ3: SIF crosses the 0.9 threshold on 8 Sep 2018, NDVI on 12 Sep, 53 and 49 days before the declaration.

What this undoes: "2018 breaks the pattern on every method", "NDVI leads in 2018, 2022, 2023", "99%+ of replicates below zero", and "NDVI crossed first in 2018". None of those survive. Same rule as Entry 17 — the paper says this openly in Section 3.3 and Section 6 instead of quietly swapping numbers. The audit also flagged that the 0.9 threshold gets crossed in early-to-mid September in wet years too, so the "7 weeks before the declaration" result is timing, not a drought alarm; the paper now says that directly.

Entry 23

1 October 2026

This was the last round on the limitations list, and it changed what the paper claims. I went through every open item one at a time and reran everything from a single script, `run_all.py`, so every number below comes from one end-to-end pass rather than a chain of scripts run by hand.

Data handling first. The GOSIF files for 2021 and 2022 had a half-pixel (0.025°) longitude offset; fixing the grid moved the regional lag by less than 0.1 day and district lags by at most 1.0 day. I had also been using two different rainfall sources for different anomalies, so I moved everything onto CHIRPS seasonal totals (1981–2023) with one 2001–2020 baseline. NDVI is now dated by the mean per-pixel `DayOfYear` of each MOD13Q1 composite and interpolated onto the SIF composite centre dates, and SIF is weighted by each cell's cropland fraction (MCD12Q1 classes 12 and 14). Together these change individual years' lags by −2.5 to +2.6 days and flip no direction; almost all of that comes from the NDVI timing, not the weighting.

Baseline sensitivity. Six study years sit inside the 2001–2020 baseline, so I reran the drought classification against 2001–2014, 1981–2014 and 1991–2020, and for every cut-off from −0.3 to −0.75. Only 2015 and 2018 are drought years in every case. Under $z < -1.0$ only 2015 is left, which is too few to compare anything.

Main lag on the corrected pipeline (threshold-crossing, mean of 0.9–0.5): 21.7, 34.2, 28.5, 8.0, 25.4, 27.1, 20.3, 11.8 and 11.1 days for 2015–2023. All nine positive; of 43 crossings, one goes the other way (2019 at 0.9, −1.3 days). The residual bootstrap keeps every year above zero, 2018's interval is [3.9, 19.8], and only 3 of 36 year-pairs have non-overlapping intervals. Cross-correlation gives 10, 12, 14, 4, 0, 9, 20, −1, −3: SIF ahead in six years, a tie in 2019, and a small NDVI lead in 2022 and 2023. The curves are flat (r at zero shift is already 0.975–0.993), and only 2015 and 2016 have intervals that exclude zero, with 2018 sitting on it.

Method dependence. I recomputed the regional lag under 40 combinations of SIF weighting, NDVI timing, smoothing and lag metric. "SIF leads in all nine years" holds in 8 of 8 mean-threshold variants and 0 of 8 cross-correlation variants. Drought years have the smaller lag in 34 of 40, and 2018 is the smallest in 22 of 40. So the direction holds for methods that read points on the decline, and not for whole-curve alignment.

TROPOMI cross-check. GOSIF is partly built from MODIS reflectance, so it isn't fully independent of NDVI. I pulled TROPOSIF v2.1 soundings for 2018–2020 (630,091 over Marathwada, 53% past the cloud filter) and built 8-day composites. TROPOMI tracks GOSIF at r = 0.91–0.97, puts SIF first in all three years and gives 2018 the smallest lead, but the lags are 12.8, 43.8 and 45.3 days against GOSIF's 8.0, 25.4 and 27.1, about 1.6–1.7 times larger. Direction and ranking carry over; the size doesn't.

District lag and H2. All 72 district-years have SIF first (median 17.7 days, range 2.4–43.6). District means run from 13.4 (Osmanabad) to 21.7 (Jalna). The two-way ANOVA gives $p = 0.055$ and the Friedman test $p = 0.043$, so H2 is only weakly supported. Cross-correlation is less consistent at district level: 52 SIF lead, 4 ties, 16 NDVI lead.

H3 and the 2018 question. Drought years average 14.8 days against 22.6 for normal years, the opposite of H3; the continuous version gives $r = 0.19$ ($p = 0.62$), and at district level it's 14.9 vs 19.7 ($p = 0.083$). I checked whether rainfall timing explains 2018's small lag (monthly totals, rainfall centroid, longest dry spell). None of them correlate with the lag at $p < 0.05$, regionally or by district. 2018 stays small and unexplained.

District seasonal SIF. District SIF on day 273 correlates with the rainfall anomaly at $r = 0.53$ ($p < 0.0001$; $p = 0.001$ with year-clustered errors), and season-mean SIF at $r = 0.74$. With district and year fixed effects it isn't significant ($p = 0.37$). Rainfall and season-mean SIF are both spatially clustered (Moran's I) in all nine years, so the clustered and fixed-effects versions are the ones I'm relying on. Osmanabad has the lowest SIF in 8 of 9 years (Bid in 2018).

Crop-yield sensitivity. The Yield Anomaly Index tracks rainfall at $r = 0.93$. Against SIF it's $r = 0.67$ ($p = 0.071$), and 0.80–0.85 once I removed zero yields and outliers or used area-weighted, detrended or major-crops-only versions. Drought years average −1.08 against +0.07 ($p = 0.15$; all variants 0.18–0.23). Right direction, not significant with two drought years.

Drought-specific signal (H4). This is the analysis I should have run much earlier. A seasonal lag only says when each index starts its normal decline, not when it leaves normal. So I standardised each year's SIF and NDVI against the normal years at each composite date and set an alarm at two consecutive composites below a z threshold from 1 July. At regional scale $z < -3$ is the only threshold with no false alarm in a normal year. In 2015 both indices fire on 24 August; in 2018 NDVI fires on 25 September and SIF on 3 October, eight days later. Across the 17 drought district-years (55 normal), both flag 12 at $z < -3$, SIF with 1 false alarm and NDVI with 5; where both fire, NDVI is earlier in 7, the same in 4 and later in 1 (median 8 days earlier). SIF's AUC reaches 0.75 on 23 July, one composite before NDVI, but NDVI's AUC is higher from 31 July to 19 October. H4 is not supported.

Rainfall Trigger-1 and declarations. I approximated the Manual for Drought Management's Trigger-1 (deficit over 25% plus a dry spell of 21 days or more, after 1 July). It never fired at region level, and fired in 4 of 17 drought district-years plus one normal one. In 3 of those 4, both satellite alarms came first (NDVI 24–86 days, SIF 16–78 days). Against declarations, both alarms came 52.5 days before the 2015 one; for the 2018 final declaration SIF was 27.5 days ahead and NDVI 35.5. But a declaration follows field verification, so a satellite signal will precede it almost by construction. The rainfall trigger is the comparison that means something.

What this changes. The paper now separates two questions I had been treating as one. SIF's seasonal decline does start before NDVI's, in every year and every district-year, and TROPOMI agrees on the direction. But SIF does not leave its normal range earlier than NDVI in a drought year; NDVI is usually first, and SIF's only edge is fewer false alarms at the strictest threshold. So SIF leads the seasonal decline but it is not an earlier drought alarm. For policy, satellite anomalies may add lead time to a rainfall trigger, but nothing here says SIF adds more than NDVI already does. I rewrote the title, abstract, highlights, discussion and conclusion around that, put the five preprint corrections into one short section (3.14), and left the remaining limitations as they are: two drought years, method dependence, lags close to the 8/16-day sampling interval, thresholds chosen during the study, GOSIF–MODIS dependence, ~5 km scale on a mixed cropland mask, non-independent districts, an approximated trigger, and declarations that are late by design.

Entry 24

Review pass and fixes. I went through the whole project again like an outside reviewer would and found a few things that change the story, so I fixed what I could without new downloads.

Normalisation. My threshold lag scales each series from its seasonal minimum to its peak. But NDVI never comes back down to its June level after the monsoon (post-peak floor 0.23–0.59 on that scale; SIF 0.00–0.28), so NDVI crosses every threshold later just because of where its floor sits. I added `normalisation_sensitivity.py`, which also scales each series from its peak to its own post-peak minimum, plus an asymmetric double-logistic end-of-season date. Decline-scaled lags: 9.6, 13.4, 10.6, 3.5, −5.5, 8.8, −1.2, −1.0, 0.4 days (2015–2023), so SIF is first in 6 of 9, not 9 of 9. Districts: 52 of 72, median 2.3 days, and the district effect disappears (Friedman $p = 0.97$ vs 0.043), so H2 is now not supported. The double-logistic dates swing from −30 to +41 days and the fits are unstable, so I only report them as a check. Cross-correlation, which does not care about scale, agrees more with the decline-scaled numbers. TROPOMI stays SIF-first under both scalings (10.4, 10.3, 27.7 days decline-scaled).

District drought baseline. District drought-years are now classified against each district's own rainfall baseline (15 drought, 57 normal) instead of the regional one (17/55), which is kept as a sensitivity check. H4 at $z < -3$: SIF flags 12, NDVI 13; where both flag, same date 6, NDVI earlier 5, SIF earlier 1; false alarms SIF 2, NDVI 4. At $z < -1.5$ NDVI is earlier in 13 of 15. H4 still not supported, and now it is the main question of the paper.

Rainfall trigger. The dry-spell counter used to run to December, so post-monsoon dry weather counted as a monsoon dry spell. Now it only counts 1 July – 30 September (the old version is kept as a variant). The trigger never fires for the region and fires in 2 of 15 drought district-years (Aurangabad 2018 on 3 Oct, Bid 2018 on 17 Sep) plus Bid 2017. I also wrote `gee_chirps_dryspell_area.py` + `rainfall_trigger_pixel.py` for a per-pixel version, which still needs to be run in Earth Engine.

Bootstrap. The threshold-lag bootstrap now resamples residuals in blocks of three and centres replicates on the reported lag. Intervals changed (2018 [2.8, 16.1]); 8 of 36 year pairs now separate; drought minus normal −7.8 days with a measurement-only interval [−9.9, +1.8].

Other checks. `ndvi_coverage_check.py`: 23 of 117 regional NDVI composites have under 25% clear cropland pixels, but dropping them changes no lag direction. `tropomi_drought_signal.py` runs H4 on TROPOMI but only has two normal years, so it is illustrative for now. Rainfall: June–September totals give the same two drought years; August 2023 was very dry ($z = -2.52$) and 40 talukas were declared in drought on 31 Oct 2023; 2001–2004 and 2014 would also be drought years, so going back before 2015 is the main way to get more drought cases. `crop_yield_validation.py` now uses season-mean SIF, the same measure as `crop_yield_sensitivity.py`: $r = 0.88$ ($p = 0.004$), and 0.80–0.85 across the sensitivity versions; the old $r =$

0.67 came from day-273 SIF alone. A full `run_all.py` rerun on my laptop (49/49 steps ok) reproduced every other number in the paper.

Figures and docs. Redid the RQ3 timeline (both drought years, alarms, trigger, declarations), the rainfall bar chart (z labels, criterion line), the anomaly curves (z = −3 line), the bootstrap and yield plots, and the graphical abstract. Map scripts now use colour scales symmetric about zero and the study-area map gets a locator inset once the state outlines are exported. Rewrote the paper around H4 as the main question, with a literature section, the normalisation result, and supplementary figures; updated the executive summary, highlights, policy brief, README, CITATION abstract, cover letter and dashboard pages to match; removed the proof-screenshot popovers and made the dashboard offer the .md documents when the PDFs are absent. Added `requirements-lock.txt` and new scripts to `run_all.py`.

Still to do (needs my machine or Earth Engine): full `run_all.py` rerun with the log committed; `gee_vi_obsdate.py` (NDVI vs EVI vs NIRv, to see whether the remaining lead is an NDVI property); the per-pixel trigger; more TROPOMI years; 2001–2014 extension; regenerating the maps; removing stale files and the decompressed raw GeoTIFFs.

Entry 25

Two checks from my finish plan, both run in the Earth Engine Code Editor and then analysed with the existing scripts.

EVI and NIRv. I wanted to know whether the small SIF lead that survives the decline-range scaling is real or just NDVI being slow. I exported EVI and NIRv (NDVI x NIR reflectance) from the same MOD13Q1 composites, same masks, same observation dates; the NDVI column matched my old NDVI file exactly. NIRv declines before NDVI in 9 of 9 years (+22.8 days season-scaled, +9.9 decline-scaled) and EVI in 9 and 8 (+17.1, +7.7), as much as SIF does (+20.9, +4.3). Compared directly, SIF is first against NIRv in only 4 of 9 years season-scaled and in none decline-scaled. So the lead is an NDVI property (saturation, slow return to baseline), not a fluorescence one. The main conclusion does not change; this answers the obvious question about NDVI.

Trigger-1 per pixel. I ran the dry-spell trigger in every CHIRPS pixel instead of on district-averaged rain. It reached 25% of a district's area only in Aurangabad 2018 (40%), Bid 2018 (35%) and Bid 2017 (72%, a normal year). In 2015 it fired in at most 7% of any district. So the trigger is rare because of the rule itself in this region, not because of averaging. Timing against the alarms is the same as before: in Aurangabad 2018 NDVI came first (8.5 days), SIF 0.5 days before; in Bid 2018 both came 0.5 days after.

Updated the paper (methods 3.4 and 3.10, results 4.6 and 4.9, discussion, limitations, conclusion, abstract, highlights), the summaries, README, CITATION, dashboard pages and the graphical abstract.

Entry 26

Out-of-sample test on 2001–2014, the last item in my finish plan. Two drought years was the biggest weakness, and 2001, 2002, 2003, 2004 and 2014 are drought years under the same rule, so I ran everything on those years without changing anything: same drought rule, same alarm rule and thresholds, same cropland weighting (each year's MCD12Q1 map), same NDVI screening and observation-day dating, same lag scalings. The only difference is that the anomalies are standardised against the normal years of 2001–2014, so nothing from 2015–2023 leaks in. I downloaded GOSIF for 2001–2014 (378 files) and exported NDVI/EVI/NIRv and cropland fractions in the Code Editor. `out_of_sample_2001_2014.py` first re-extracted 2015 and got the

main series back exactly (largest difference about 1e-16), and `--check` mode reproduces every main alarm and lag exactly.

What came out. Districts (40 drought, 72 normal district-years): the same picture as 2015–2023. Where both flagged a drought district-year, NDVI was earlier in 22 of 37 at z < −1, 17 of 22 at z < −1.5 and all 13 at z < −2; SIF earlier in 10, 1 and 0. SIF again had fewer false alarms (16 vs 25 at z < −1.5). Region (5 drought, 9 normal years): the vegetation anomalies were weaker than in 2015 and 2018, and here SIF came out slightly ahead. At z < −3 SIF flagged 2001 (17 Sep) and 2002 (25 Sep), NDVI only 2002 on the same date, and no normal year was flagged by either; at z < −1.5 SIF was first in four drought years and NDVI in none. Over both periods SIF flags four of seven drought years at z < −3 and NDVI three, coinciding twice with NDVI earlier once, and at district level NDVI is earlier far more often. H4 stays not supported, but the regional evidence is less one-sided than in 2015–2023.

Seasonal lags reproduce: season-scaled SIF leads NDVI in 14 of 14 years (+27.4 days) and NIRv leads NDVI by almost the same (+26.6); decline-scaled NIRv declines before SIF in 11 of 14 years (−6.4 days). The lead is not larger in drought years (+28.3 vs +26.9 season-scaled), so H3 is not supported with five drought years either.

I kept the extracted 2001–2014 SIF series in `data/processed/oos_sif_*.csv` and added `data/raw/*.tif.gz` to `.gitignore`, so the GOSIF archives stay out of the repository and the test still reruns from the CSVs. Updated the paper (new sections 3.14 and 4.11 with Tables 5 and 6, abstract, highlights, Table 1, discussion, limitations, conclusion, revision note), the executive summary, policy brief, README, CITATION, cover letter and the dashboard.

Entry 27

A final check of the whole repository before export, and one real error.

The district alarm comparison was dating every district NDVI alarm one composite too early. In `drought_signal_analysis.py` the district SIF and NDVI series go into two `pivot_table`s, one row per district-year and one column per SIF composite date. No district has an NDVI observation before the first SIF composite centre (day 156.5; the earliest NDVI observation day is 161.4), so that column is empty in the NDVI table and `pivot_table` dropped it. The SIF table kept 27 columns, the NDVI table had 26, and both were dated with the same 27 composite dates, so each NDVI anomaly was read one composite (8 days) before its real date. I checked it by hand on Aurangabad 2018: the NDVI anomaly first stays below −3 on day 260.5, but the table said 252.5. The out-of-sample script had the same line. The regional series were not affected, because they go through `pivot`, which keeps empty columns. The fix keeps every composite date in both tables (`dropna=False` and `reindex`) and asserts that the columns match.

What changed. Districts, 2015–2023, z < −3: SIF flags 12 and NDVI 13 as before, but where both flag, SIF is earlier in 7, the same in 2 and NDVI earlier in 3 (it was 1, 6 and 5). At z < −1.5 it is 2, 4 and 9 (was 1, 1 and 13); at z < −2, 4, 4 and 6 (was 2, 2 and 10). In 11 of the 12 at z < −3 the difference is at most one composite. With the regional baseline, z < −3 gives 5, 4 and 3. The date-by-date AUC now reaches 0.75 on 23 July for SIF and 31 July for NDVI. Out of sample, at z < −1.5 it is 5, 11 and 6 (was 1, 4 and 17), and at z < −2, 0, 7 and 6 (was 0, 0 and 13). Rainfall trigger: in Aurangabad 2018 both alarms came 15.5 days before the trigger; in Bid 2018 SIF came 0.5 days after it and NDVI 8.5 days after. Hit and false-alarm counts, all regional alarms and every seasonal lag are unchanged.

So the district result is no longer "NDVI was usually first". The two alarms are mostly within one composite of each other, with no consistent winner. Since NDVI is a 16-day product interpolated onto 8-day SIF dates, I also

wrote `alarm_dating_sensitivity.py`, which reruns the alarm test with NDVI taken as observed (the last NDVI value available by the end of each SIF composite) instead of interpolated, because the interpolated value already contains part of the next NDVI observation. With that dating SIF is first in both regional drought years at $z < -3$ (by 8 days each) and in 8 of 12 district-years. The direction of a one-composite difference therefore depends on a dating choice, and I now report H4 as not supported in the sense that neither index gives a consistently earlier alarm the data can resolve.

TROPOMI against NIRv. My paper said that without a TROPOMI comparison against NIRv I could not separate a fluorescence lead from an NDVI effect, but I already had both series. `tropomi_vs_reflectance.py` computes it: TROPOMI declines before NIRv in all three years (5.4, 7.8 and 13.1 days season-scaled; 4.6, 1.2 and 12.5 decline-scaled) and before EVI by 3.0–20.8 days, and in 23 of 24 year-and-scaling cases across the four TROPOMI variants. GOSIF not leading NIRv is partly built in, since GOSIF is predicted from MODIS reflectance. Three years are too few to call this a fluorescence lead, but they are enough that I can no longer write that the whole lead is an NDVI effect. The paper now says this for GOSIF only and reports the TROPOMI result as open.

Smaller fixes from the same check: the cut-off statement now says the -0.3 to -0.75 range holds for the main 2001–2020 baseline (with the 1981–2014 baseline, 2018 drops out at -0.7); the cross-correlation is described as running over the post-peak decline, not the whole curve; the out-of-sample text says no vegetation data from 2015–2023 enter the test, since the drought rule keeps its 2001–2020 rainfall baseline; and the README says the GOSIF files need to be downloaded once before `run_all.py`. Updated the paper (abstract, highlights, 3.9, 3.11, 3.14, 4.1, 4.2 with Table 2, 4.6, 4.9, 4.11 with Table 5, discussion, limitations, conclusion, revision note), the executive summary, CITATION, cover letter, policy brief, README, dashboard, graphical abstract and both portfolio pages.